Traitement et analyse de données géospatiales avec R

Des packages aux inégalités territoriales
d'accès à l'enseignement supérieur en Normandie

Rapport de projet

Master 1 Mathématiques appliquées et statistiques

présenté par

Amandine GBAYE

Ange-Noella Kaneza

Sous la direction de : M. Christophe Chesneau

Soutenance : Juillet 2026

Université de Caen Normandie • Année universitaire 2025–2026

Table des matières

Chapitre 1 : Les données géospatiales **4**
1.1 Qu'est-ce qu'une donnée géospatiale ? . . . 4
1.1.1 Une valeur et une localisation . . . 4
1.1.2 Pourquoi des méthodes spécifiques ? . . . 4
1.2 Trois grands types de données spatiales . . . 4
1.2.1 Les données ponctuelles . . . 4
1.2.2 Les données continues . . . 4
1.2.3 Les données surfaciques . . . 4
1.3 Deux modèles de représentation : vectoriel et raster . . . 5
1.3.1 Le modèle vectoriel . . . 5
1.3.2 Le modèle raster . . . 5
1.4 Se repérer sur la Terre : les systèmes de coordonnées . . . 5
1.4.1 Les systèmes géographiques . . . 5
1.4.2 Les systèmes projetés . . . 6
1.5 Les formats de fichiers . . . 6
1.6 La sémiologie cartographique . . . 6
1.6.1 Les variables visuelles . . . 6
1.6.2 À chaque indicateur son type de carte . . . 6
1.6.3 La discrétisation . . . 7
1.6.4 Le choix des couleurs . . . 7
1.6.5 Les règles d'habillage . . . 7
1.7 De la théorie aux packages R . . . 8

Chapitre 2 : Manipuler les données vectorielles avec sf **9**
2.1 De sp à sf : un bref rappel historique . . . 9
2.2 Le package sf : le standard moderne . . . 9
2.2.1 Rôle et notions essentielles . . . 9
2.2.2 Les fonctions clés . . . 9
2.2.3 Applications . . . 12

Chapitre 3 : Cartographier, ggplot2, tmap, leaflet **15**
3.1 Le package ggplot2 . . . 15
3.1.1 Rôle . . . 15
3.1.2 Les fonctions clés . . . 15
3.1.3 Applications . . . 19
3.2 Le package tmap . . . 21
3.2.1 Rôle . . . 21
3.2.2 Les fonctions clés . . . 21
3.2.3 Applications . . . 24
3.3 Le package leaflet . . . 25
3.3.1 Rôle . . . 25
3.3.2 Les fonctions clés . . . 26
3.3.3 Applications . . . 27

Chapitre 4 : Traiter les données raster, terra et stars **28**
4.1 Le package terra : le successeur de raster . . . 28
4.1.1 Rôle . . . 28
4.1.2 Les fonctions clés . . . 28
4.1.3 Applications . . . 33
4.2 Le package stars : raster et cubes de données spatio-temporels . . . 37
4.2.1 Rôle . . . 37
4.2.2 Les fonctions clés . . . 37

4.2.3 Applications . . . 39

Chapitre 5 : Analyser les structures spatiales, spdep et gstat **42**
5.1 spdep : l'autocorrélation spatiale . . . 42
5.1.1 Rôle . . . 42
5.1.2 Les fonctions clés . . . 42
5.1.3 Applications . . . 45
5.2 gstat : interpolation et géostatistique . . . 46
5.2.1 Rôle . . . 47
5.2.2 Les fonctions clés . . . 47
5.2.3 Applications . . . 49

Chapitre 6 : Cartographier et analyser l'offre de formation en Normandie **52**
Pourquoi cette étude . . . 52
Ce que disent les travaux existants . . . 52
Les données et le terrain . . . 53
Préparer les données avec sf . . . 53
Importer et filtrer . . . 53
Charger et assembler le fond communal . . . 53
Reprojeter en Lambert 93 . . . 54
Joindre les formations aux communes . . . 54
Agréger l'offre par commune . . . 54
Cartographier l'offre avec tmap . . . 55
Analyser la structure spatiale avec spdep . . . 57
Construire la matrice de voisinage . . . 57
L'indice de Moran global . . . 57
Les clusters locaux : LISA . . . 58
Cartographier les regroupements . . . 59
Ce que révèle l'analyse . . . 60

Conclusion générale **61**

Bibliographie **62**
Ouvrages et manuels de référence . . . 62
Accès territorial à l'enseignement supérieur . . . 62
Sources de données . . . 62
Packages R (documentation officielle) . . . 62
Fondamentaux méthodologiques . . . 63

Annexe : Code des fonctions secondaires **64**
A.1 Fonctions sf secondaires . . . 64
Fusion de polygones avec st_union() . . . 64
Buffer et autres opérations géométriques . . . 64
A.2 Fonctions ggplot2 secondaires pour la cartographie . . . 65
Étiquetage avec geom_sf_text() . . . 65
Palettes de couleurs avec viridis . . . 66
A.3 Fonctions tmap secondaires . . . 67
Habillage complet avec tm_layout() . . . 67
Cercles proportionnels . . . 68
A.4 Fonctions leaflet secondaires . . . 69
Palette de couleurs et légende . . . 69
A.5 Fonctions terra . . . 69
Statistiques de raster . . . 70
A.6 Fonctions spdep . . . 70
Voisinage Rook et k-plus proches voisins . . . 70

Poids spatiaux alternatifs 71
A.7 Gstat et géostatistique 71
Variogramme et diagnostic 72
Krigeage simple et universel 72

Table des figures

1 Contours des comtés de Caroline du Nord. 13
2 Zone tampon de 10 km autour du premier comté. 14
3 Comtés de Caroline du Nord colorés selon leur superficie (ggplot2). 16
4 Naissances de 1974 par comté, dégradé jaune-rouge. 17
5 Superficie des comtés avec titre et thème épuré. 18
6 Taux de mortalité infantile et contour de l'État (ggplot2). 19
7 Comparaison des taux de mortalité infantile, 1974 et 1979 (ggplot2). 20
8 Contours des comtés (tmap). 21
9 Naissances de 1974, découpage automatique (tmap). 22
10 Naissances de 1974 en cinq classes, palette bleu-vert. 23
11 Mortalité infantile en cinq classes Jenks (tmap). 24
12 Naissances de 1974 en cercles proportionnels (tmap). 25
13 Altitude du Luxembourg, en mètres. 29
14 Altitude reclassée en trois classes. 31
15 Altitude du district de Diekirch après découpage. 32
16 Altitudes du Luxembourg réparties en trois classes. 34
17 Découpage par crop seul (gauche) et par crop puis mask (droite). 36
18 Les six bandes spectrales de l'image Landsat. 38
19 Première bande de l'image Landsat. 40
20 Indice de végétation NDVI ; le vert foncé marque la végétation dense. 41
21 Diagramme de Moran du taux de mortalité infantile. 44
22 Semi-variogramme expérimental du zinc (échelle log). 48
23 Offre de formation Parcoursup par commune, Normandie 2024 56
24 Nombre de formations par commune (cercles proportionnels) 57
25 Clusters LISA de l'offre de formation, Normandie 2024 60

Chapitre 1 : Les données géospatiales

1.1 Qu'est-ce qu'une donnée géospatiale ?

1.1.1 Une valeur et une localisation

Une donnée géospatiale est une observation dont on connaît deux choses à la fois : sa valeur et l'endroit où elle a été relevée. Le prix d'un appartement est une donnée classique. Le prix d'un appartement situé au 12 rue Écuyère à Caen devient une donnée géospatiale. Cette information de localisation n'est pas un simple détail administratif. Elle change profondément la manière dont on peut analyser les données.

1.1.2 Pourquoi des méthodes spécifiques ?

Les observations localisées ne sont presque jamais indépendantes les unes des autres. Le prix de cet appartement de la rue Écuyère ressemble beaucoup à celui de l'immeuble voisin, un peu moins à celui d'un logement de la périphérie, et plus du tout à celui d'une ferme de l'Orne. Ce constat, souvent résumé par la première loi de la géographie de Tobler, selon laquelle tout interagit avec tout mais deux choses proches interagissent davantage que deux choses éloignées, pose un vrai problème statistique. Deux propriétés des données spatiales contredisent en effet les hypothèses des méthodes classiques :

- **la dépendance spatiale** : la valeur observée en un lieu influence celle des lieux voisins. L'hypothèse d'indépendance des observations, au cœur de la régression linéaire par exemple, n'est donc plus tenable ;
- **l'hétérogénéité spatiale** : une relation entre deux variables peut être forte dans une région et faible, voire inversée, dans une autre. L'effet de la distance à la gare sur les prix immobiliers n'est sans doute pas le même à Rouen et dans le Cotentin.

Ces deux particularités justifient l'existence de méthodes d'analyse propres aux données spatiales, et donc de packages R spécialisés pour les mettre en œuvre.

1.2 Trois grands types de données spatiales

La typologie la plus répandue, due à Cressie, distingue les données spatiales selon le processus qui les génère et non selon la taille des unités géographiques considérées (Loonis et de Bellefon, 2018). Cette nuance est importante car elle détermine la famille de méthodes à mobiliser.

1.2.1 Les données ponctuelles

Ici, c'est la localisation elle-même qui constitue l'information. Aucune valeur n'est mesurée, seule compte la position des événements dans l'espace. Les lieux de cambriolages signalés dans une ville en sont un bon exemple : la question n'est pas de savoir combien d'objets ont été volés, mais si les cambriolages se concentrent dans certains quartiers ou se répartissent au hasard. L'analyse de ces configurations de points cherche précisément à mesurer cet écart par rapport au hasard.

1.2.2 Les données continues

La variable étudiée possède une valeur en tout point du territoire, mais on ne la mesure qu'en un nombre limité d'endroits. La température illustre bien cette situation : il fait une certaine température partout en Normandie à cet instant, pourtant on ne dispose de relevés que dans quelques stations météorologiques. Tout l'enjeu consiste alors à prédire la valeur de la variable là où elle n'a pas été mesurée. C'est le domaine de la géostatistique, avec ses outils emblématiques que sont le variogramme et le krigeage.

1.2.3 Les données surfaciques

Cette fois, les localisations sont fixes et connues d'avance, généralement un découpage du territoire en zones contiguës, et c'est la valeur attachée à chaque zone qui résulte d'un processus aléatoire. Le taux de chômage par commune ou le nombre de naissances par département entrent dans cette catégorie. Le terme surfacique

est d'ailleurs un peu trompeur puisque ces données peuvent aussi être rattachées à des points fixes, comme des établissements scolaires. L'analyse porte sur les relations entre zones voisines : une commune au taux de chômage élevé est-elle entourée de communes dans la même situation ? C'est avec ce type de données que nous travaillerons en seconde partie, les indicateurs d'accès à l'enseignement supérieur étant agrégés par département et par commune.

1.3 Deux modèles de représentation : vectoriel et raster

Une fois la nature des données identifiée, encore faut-il les représenter dans un format que l'ordinateur sait manipuler. Deux grands modèles coexistent et structurent l'ensemble de l'écosystème R spatial.

1.3.1 Le modèle vectoriel

Le modèle vectoriel décrit les objets géographiques par leurs coordonnées. Trois géométries de base suffisent à presque tout représenter :

- **le point**, pour les objets dont l'emprise au sol importe peu à l'échelle de l'étude : une pharmacie, un arrêt de bus, un lycée ;
- **la ligne**, simple suite ordonnée de points reliés entre eux, pour les routes, les rivières ou les voies ferrées ;
- **le polygone**, ligne fermée délimitant une surface, pour les communes, les parcelles ou les lacs.

Ces géométries existent aussi en version multiple (multipoint, multiligne, multipolygone), ce qui permet par exemple de traiter une commune composée de plusieurs morceaux disjoints comme un seul objet. Le standard Simple Features, adopté par le package sf que nous présenterons au chapitre suivant, formalise précisément ces types de géométries (Pebesma et Bivand, 2023).

La force du modèle vectoriel tient à l'association entre les géométries et une table attributaire. Chaque ligne de la table correspond à un objet géographique et chaque colonne à une caractéristique : la couche des communes normandes contient ainsi, pour chaque polygone, un code Insee, un nom, une population. On retrouve la définition donnée plus haut, une valeur et une localisation réunies dans une même structure.

1.3.2 Le modèle raster

Le modèle raster procède autrement. Le territoire est découpé en une grille régulière de cellules, le plus souvent carrées, et chaque cellule porte une valeur. Une photographie satellite est un raster dont les cellules contiennent des intensités de couleur. Un modèle numérique de terrain est un raster dont chaque cellule indique une altitude.

La notion centrale est ici la résolution, c'est-à-dire la taille d'une cellule sur le terrain. Avec une résolution de 25 mètres, une cellule couvre 625 mètres carrés et tout détail plus fin disparaît. Le choix entre vectoriel et raster dépend donc du phénomène étudié : les objets discrets aux contours nets se prêtent au vectoriel, les phénomènes qui varient continûment dans l'espace, comme l'altitude ou l'occupation du sol, se prêtent au raster.

1.4 Se repérer sur la Terre : les systèmes de coordonnées

Toute coordonnée n'a de sens que par rapport à un système de référence, et la confusion entre systèmes est sans doute la source d'erreurs la plus fréquente chez les débutants. Deux couches parfaitement valides peuvent refuser de se superposer simplement parce qu'elles ne parlent pas le même langage de coordonnées.

1.4.1 Les systèmes géographiques

Les systèmes géographiques expriment une position par une longitude et une latitude, en degrés, sur un modèle mathématique de la Terre appelé ellipsoïde. Le plus connu est le WGS 84, celui des GPS et des applications de cartographie en ligne. Ces coordonnées ont un défaut sérieux pour le calcul : un degré de longitude ne représente pas la même distance partout. Il vaut environ 111 kilomètres à l'équateur mais à

peine 73 kilomètres à la latitude de Caen. Calculer une distance ou une surface directement en degrés conduit donc à des résultats faux.

1.4.2 Les systèmes projetés

C'est pourquoi on recourt aux systèmes projetés, qui transforment la surface courbe de la Terre en un plan où les coordonnées s'expriment en mètres. Aucune projection ne préserve à la fois les angles, les distances et les surfaces, chacune sacrifie quelque chose. Chaque pays adopte donc une projection adaptée à son territoire. En France métropolitaine, la référence légale est le Lambert 93.

Pour s'y retrouver parmi les milliers de systèmes existants, on utilise les codes EPSG, des identifiants numériques uniques. Deux codes suffisent à couvrir la plupart des besoins d'une étude sur la France :

- **EPSG 4326** : le WGS 84, en degrés, pour les données issues du GPS ou du web ;
- **EPSG 2154** : le Lambert 93, en mètres, pour tout calcul de distance ou de surface.

Concrètement, avant toute mesure sur des données françaises, le réflexe à acquérir est de reprojeter les couches en Lambert 93.

1.5 Les formats de fichiers

Les données spatiales circulent sous plusieurs formats, dont quatre couvrent l'essentiel des situations :

- **le shapefile**, le plus ancien et encore le plus répandu, présente une particularité déroutante : un même fichier est en réalité un ensemble d'au moins trois fichiers portant le même nom mais des extensions différentes. Le .shp stocke les géométries, le .dbf contient la table attributaire et le .shx sert d'index. Il suffit d'en oublier un lors d'une copie pour que la couche devienne illisible, mésaventure classique lorsqu'on télécharge des données en vrac ;
- **le GeoPackage** corrige ce défaut en regroupant tout dans un fichier unique, et lève les limites du shapefile sur la longueur des noms de champs ou la taille des fichiers ;
- **le GeoJSON**, un simple fichier texte, s'est imposé pour l'échange de données vectorielles sur le web ;
- **le GeoTIFF** domine largement côté raster : il s'agit d'une image TIFF enrichie des informations de géoréférencement.

1.6 La sémiologie cartographique

Représenter des données spatiales aboutit presque toujours à une carte. Or une carte n'est pas une simple illustration, c'est un langage à part entière, avec sa grammaire et ses règles. Cette grammaire porte un nom : la sémiologie cartographique. Elle désigne l'ensemble des règles qui permettent de transmettre une information juste, de la façon la plus claire possible, à travers une image cartographique. Les ignorer expose à un risque réel, celui de produire une carte fausse sans le savoir, car le lecteur croit ce qu'il voit.

1.6.1 Les variables visuelles

Le vocabulaire de ce langage repose sur les variables visuelles : la forme, la taille, la couleur, la texture et l'orientation des objets représentés, auxquelles s'ajoute aujourd'hui l'animation des cartes interactives. Chaque variable a sa spécialité :

- **la taille** traduit des quantités ;
- **la couleur** exprime une hiérarchie de valeurs ou des différences de nature ;
- **la forme** distingue des catégories d'objets.

Bien combinées, ces variables renforcent le message ; mal choisies, elles le brouillent.

1.6.2 À chaque indicateur son type de carte

La règle fondamentale relie le type d'indicateur à son mode de représentation :

- **un effectif brut** (volume, stock) se représente par des cercles proportionnels : le nombre de candidats Parcoursup par commune, par exemple, donne des ronds dont la surface varie avec la valeur ;
- **un ratio, un taux, une part ou une évolution** appelle une carte en aplats de couleurs, dite choroplèthe : un taux d'admission ou une part de boursiers se prête à ce traitement ;
- **un flux entre territoires**, comme des migrations d'étudiants, se traduit par des flèches proportionnelles ;
- **une simple localisation** d'équipements se contente de symboles ponctuels.

Pourquoi cette règle est-elle si stricte ? Parce que représenter un effectif en aplats de couleurs trompe mécaniquement l'œil du lecteur, qui associe la quantité à la surface coloriée. Imaginons une carte des communes normandes où le nombre d'étudiants serait traduit par des couleurs. Une vaste commune rurale de l'Orne, même peu peuplée, occuperait visuellement plus de place qu'une petite commune urbaine dense de l'agglomération rouennaise. Le lecteur en déduirait une géographie étudiante inversée.

L'inverse mérite aussi la vigilance : un taux très élevé peut ne concerner qu'une poignée d'individus. Il est alors judicieux de combiner les deux approches, des cercles proportionnels aux effectifs coloriés selon le taux, pour donner à voir le volume et l'intensité sur une même carte.

1.6.3 La discrétisation

Une carte choroplèthe suppose de regrouper les valeurs en classes, opération appelée discrétisation. Deux questions se posent : combien de classes, et avec quelles bornes ?

Pour le nombre de classes, la règle de Sturges propose la formule $1 + 3{,}3 \log10(N)$, où N est le nombre d'observations. En pratique, on retient souvent trois classes en dessous de cinquante observations, quatre entre cinquante et cent cinquante, cinq au-delà, avec une classe supplémentaire quand la variable comporte des valeurs négatives et positives.

Pour les bornes, plusieurs méthodes coexistent et aucune n'est parfaite :

- **les quantiles** placent le même nombre d'observations dans chaque classe. La carte obtenue est équilibrée et lisible, mais des valeurs presque identiques peuvent se retrouver dans des classes différentes ;
- **les amplitudes égales** découpent l'étendue des valeurs en intervalles de même longueur. La méthode est simple à expliquer, mais elle laisse parfois des classes vides quand la distribution est asymétrique ;
- **la méthode de Jenks** construit des classes aussi homogènes que possible, en maximisant la variance entre les classes et en la minimisant à l'intérieur de chacune. Le calcul devient toutefois lourd sur de gros volumes de données ;
- **la méthode des k-means** poursuit le même objectif avec un calcul plus rapide, mais son résultat peut varier d'une exécution à l'autre, ce qui oblige à la répéter plusieurs fois pour retenir la meilleure partition ;
- **le découpage manuel** consiste à fixer soi-même les bornes. Il est précieux pour faire ressortir des seuils qui ont du sens, comme une borne à zéro pour une évolution, ou pour imposer les mêmes classes à plusieurs cartes que l'on veut comparer.

Une démarche raisonnable consiste à partir d'une classification de Jenks puis à arrondir les bornes à la main.

1.6.4 Le choix des couleurs

Les couleurs obéissent elles aussi à des conventions. Un dégradé d'une même teinte traduit une hiérarchie de valeurs, les tons les plus foncés signalant les valeurs les plus fortes. Quand la variable oppose des valeurs positives et négatives, on associe les teintes chaudes comme le rouge ou l'orange aux premières et les teintes froides comme le bleu aux secondes, la rupture de couleur marquant le passage par zéro.

1.6.5 Les règles d'habillage

Quelques principes complètent ces règles et s'appliquent à toute carte, quel que soit son contenu :

- **un seul message par carte** : vouloir tout montrer sur une même image garantit que le lecteur n'en retiendra rien ;

- **les mentions obligatoires** : un titre informatif, une légende, l'indication du zonage représenté et la source des données. L'échelle et la flèche du nord restent facultatives ;
- **ne pas présenter le territoire comme une île** : faire figurer les régions voisines, la mer ou quelques villes repères aide le lecteur à se situer. Une carte de la Normandie gagne ainsi à laisser apparaître l'Île-de-France voisine, dont l'attraction pèse précisément sur les poursuites d'études ;
- **harmoniser les cartes comparées** : deux cartes destinées à être lues côte à côte doivent partager les mêmes classes, la même échelle et le même cadrage, faute de quoi la comparaison visuelle n'a aucune valeur.

1.7 De la théorie aux packages R

Ces notions trouvent leur traduction directe dans l'écosystème R. Aux données vectorielles correspondent historiquement le package sp, puis son successeur sf qui s'est imposé depuis quelques années. Aux données raster correspondent le package raster, puis terra qui le remplace progressivement. Cette transition n'est pas anecdotique : elle a redessiné tout l'écosystème spatial de R, notamment après le retrait de packages fondateurs comme rgdal. Le chapitre suivant retrace cette évolution et présente en détail la manipulation des objets sf, brique de base de tout le reste du rapport.

Chapitre 2 : Manipuler les données vectorielles avec sf

Le chapitre précédent a posé les concepts ; celui-ci entre dans R. La première étape de la chaîne de traitement consiste à charger des données vectorielles et à les manipuler. Le package de référence pour cela est aujourd'hui **sf**. Avant d'y venir, un bref rappel historique sur son prédécesseur, sp, permet de comprendre d'où vient sf et pourquoi il s'est imposé.

On parlera souvent de **couche** dans ce chapitre. Le terme désigne un ensemble d'objets géographiques de même nature rangés ensemble, comme la couche des communes ou la couche des routes. Il vient des logiciels de cartographie, où l'on superpose ces ensembles comme des feuilles de papier calque.

Pour illustrer les fonctions, nous utilisons un jeu de données livré avec le package, ce qui rend les exemples reproductibles sans téléchargement : le shapefile des 100 comtés de Caroline du Nord, fourni avec sf.

2.1 De sp à sf : un bref rappel historique

Pendant près de quinze ans, manipuler des données spatiales dans R passait par le package **sp**, publié en 2005 par Edzer Pebesma et Roger Bivand. Avant lui, chaque package inventait ses propres conventions ; sp a apporté un système de classes unifié pour représenter points, lignes et polygones, et la plupart des outils de l'époque (rgdal, rgeos, raster) se sont construits autour de lui.

sp repose sur le système de classes S4 : un objet y est une boîte compartimentée, dont chaque compartiment, appelé *slot*, contient une information précise, à laquelle on accède avec l'opérateur `@` plutôt que le `$` habituel. Cette logique, peu compatible avec le tidyverse et répartie entre plusieurs packages, a fini par montrer ses limites. En octobre 2023, rgdal et rgeos ont été retirés du CRAN, et sp est entré en maintenance passive.

Aujourd'hui, **sf** est le standard pour tout nouveau projet, et c'est lui que nous présentons en détail. sp garde toutefois un intérêt pratique : de nombreux packages d'analyse reposent encore sur ses objets, et l'on doit parfois convertir une couche d'un format à l'autre. Cette conversion se fait en une ligne, dans un sens comme dans l'autre, avec `st_as_sf()` et `as()`.

2.2 Le package sf : le standard moderne

2.2.1 Rôle et notions essentielles

Publié en 2016, sf repose sur une idée simple : implémenter dans R la norme internationale **Simple Features**, déjà adoptée par les bases de données spatiales et les SIG. Cette norme définit dix-sept types de géométries, dont sept couvrent la quasi-totalité des usages : le point, la ligne et le polygone, leurs versions multiples, et la collection de géométries.

La rupture avec sp est radicale : un objet sf est un data.frame ordinaire, dont une colonne contient les géométries. Plus de slots, plus d'opérateur `@`. Trois niveaux d'objets s'emboîtent :

- `sfg` : une géométrie individuelle, par exemple un point ;
- `sfc` : la colonne qui rassemble toutes les géométries de la couche, et qui porte le système de coordonnées ;
- `sf` : le tableau complet, attributs et colonne géométrique réunis.

Cette architecture a une conséquence pratique considérable : tout ce qui fonctionne sur un data.frame fonctionne sur un objet sf, y compris les verbes de dplyr comme `filter()` ou `mutate()`. Le package s'intègre ainsi naturellement au tidyverse. Autre convention agréable, toutes ses fonctions commencent par le préfixe `st_` (pour spatial type), ce qui facilite l'autocomplétion.

2.2.2 Les fonctions clés

Les fonctions de sf se regroupent par usage, de l'import des données à leur croisement. On présente ci-dessous les principales, avec un exemple de code pour celles qui sont au cœur de la chaîne de traitement ; le code des fonctions plus secondaires figure en annexe.

2.2.2.1 Importer une couche : `st_read()`

Pour charger une couche vectorielle dans R, la fonction d'entrée est `st_read()`. Elle lit la plupart des formats géographiques (shapefile, GeoJSON, GeoPackage…) en s'appuyant sur la bibliothèque GDAL. On lui donne le chemin du fichier, et elle renvoie un objet sf prêt à l'emploi.

Trois arguments suffisent dans la plupart des cas :

- `dsn` désigne la source des données, le plus souvent le chemin vers le fichier ;
- `layer` précise la couche à lire quand la source en contient plusieurs, comme dans un GeoPackage ; on peut l'omettre lorsqu'il n'y en a qu'une ;
- `quiet` contrôle le résumé affiché à la lecture : en le passant à `TRUE`, on supprime ce message, utile dans un document compilé pour qu'il n'apparaisse pas dans le résultat final.

```
library(sf)
nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
```

À la lecture, `st_read()` récupère automatiquement le système de coordonnées s'il est défini dans le fichier, ce qui évite d'avoir à le redéclarer.

2.2.2.2 Construire un objet spatial : `st_as_sf()`

Les données spatiales arrivent parfois sous forme de tableau, les coordonnées rangées dans des colonnes. Il faut alors les convertir en objet spatial avant toute analyse. C'est le rôle de `st_as_sf()`, qui crée une géométrie de points à partir de ces colonnes.

Deux arguments sont déterminants :

- `coords` désigne les deux colonnes contenant les coordonnées, dans l'ordre x puis y, soit la longitude d'abord et la latitude ensuite ;
- `crs` déclare le système de coordonnées de ces valeurs, par son code EPSG ; pour des coordonnées GPS, c'est le WGS 84, de code 4326.

```
villes <- data.frame(
  nom = c("Caen", "Rouen", "Le Havre"),
  longitude = c(-0.37, 1.10, 0.11),
  latitude = c(49.18, 49.44, 49.49)
)
villes_sf <- st_as_sf(villes, coords = c("longitude", "latitude"), crs = 4326)
```

L'ordre des colonnes dans `coords` n'est pas anodin : la longitude vient avant la latitude, à l'inverse de l'ordre habituel à l'oral, car la convention géographique place x avant y.

2.2.2.3 Reprojeter : `st_transform()`

Une couche est rarement dans le bon système de coordonnées dès le départ. Pour la reprojeter, on emploie `st_transform()`, qui recalcule toutes les coordonnées dans le système demandé. À la différence de `st_crs()`, qui ne fait que déclarer le système, `st_transform()` transforme réellement les positions.

La fonction repose sur :

- `x`, la couche à reprojeter ;
- `crs`, le système cible, indiqué par son code EPSG.

```
villes_l93 <- st_transform(villes_sf, crs = 2154)
```

Ici, les points passent du WGS 84 (degrés) au Lambert 93 (mètres), de code 2154. Cette étape est indispensable avant tout calcul de distance ou de surface, car ces mesures n'ont de sens que dans un système exprimé en mètres.

2.2.2.4 Résumer un objet par son centre : `st_centroid()`

Certaines analyses ont besoin de résumer chaque objet par un point unique, son centre. C'est le rôle de `st_centroid()`, qui calcule le centre géométrique de chaque entité d'une couche. Un polygone de département, par exemple, devient un point situé en son milieu.

```
nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
centres <- st_centroid(nc)
```

Chaque comté est désormais représenté par un point unique. Le centroïde sert notamment à placer une étiquette au milieu d'une zone, ou à mesurer des distances entre zones, le calcul se faisant alors de centre à centre.

2.2.2.5 Croiser des couches : `st_join()`

Croiser deux couches selon leur position dans l'espace est l'une des opérations les plus utiles de l'analyse spatiale. C'est la jointure spatiale, réalisée par `st_join()`. Là où une jointure ordinaire relie deux tableaux par une colonne commune, la jointure spatiale les relie par leur géométrie : pour chaque objet de la première couche, elle cherche l'objet de la seconde avec lequel il coïncide, et lui ajoute ses attributs.

Le cas le plus courant croise une couche de points avec une couche de polygones. Par exemple, à partir d'une couche d'écoles et d'une couche de communes, la jointure attribue à chaque école le nom de la commune dans laquelle elle se trouve. L'appariement est défini par une relation spatiale, par défaut « se recoupe » (`st_intersects`), qui convient dans la plupart des cas. Le code détaillé d'une telle jointure figure en annexe.

2.2.2.6 Les autres fonctions utiles

D'autres fonctions de sf complètent la chaîne de traitement ; leur code figure en annexe.

Pour l'export, `st_write()` enregistre une couche dans un fichier, dans le format déduit de l'extension choisie. Elle fait l'opération inverse de `st_read()` : l'une lit un fichier, l'autre l'enregistre.

Pour examiner une couche, `st_crs()` consulte ou déclare son système de coordonnées, `st_geometry()` isole la colonne des géométries, et `st_drop_geometry()` fait l'inverse en ne gardant que la table attributaire, c'est-à-dire le tableau des données sans la dimension spatiale.

Pour mesurer, `st_area()` calcule la surface des polygones et `st_distance()` les distances entre objets. Ces mesures supposent une couche projetée, dont les coordonnées sont exprimées en mètres et non en degrés.

Pour fabriquer de nouvelles géométries, `st_buffer()` trace une zone tampon autour d'un objet, c'est-à-dire une zone qui l'entoure jusqu'à une distance donnée, par exemple les 500 mètres autour d'une école. `st_union()` fusionne plusieurs objets en un seul, comme des communes réunies en une région. Et `st_intersects()` indique quels objets se recoupent, c'est-à-dire se touchent ou se chevauchent ; cette fonction sert aussi de relation par défaut dans la jointure spatiale.

Le tableau suivant récapitule l'ensemble des fonctions présentées :

Usage	Fonction	Rôle
Importer, exporter	`st_read()`, `st_write()`	lire et enregistrer une couche
Construire un objet spatial	`st_as_sf()`	transformer un tableau de coordonnées en couche de points
Gérer les coordonnées	`st_crs()`, `st_transform()`	consulter le système, reprojeter
Accéder aux deux faces	`st_geometry()`, `st_drop_geometry()`	isoler les géométries ou la table attributaire
Mesurer	`st_area()`, `st_distance()`	surfaces et distances
Construire des géométries	`st_centroid()`, `st_buffer()`, `st_union()`	centres, zones tampons, fusion
Croiser des couches	`st_join()`, `st_intersects()`	jointure spatiale et relations

2.2.3 Applications

Application 1. On dispose du jeu de données des comtés de Caroline du Nord, fourni avec sf. On veut le prendre en main avant toute analyse.

1. Importez la couche avec `st_read()` ;
2. consultez son système de coordonnées avec `st_crs()` ;
3. isolez sa table attributaire avec `st_drop_geometry()` et affichez ses premières lignes ;
4. tracez une première carte de la couche.

Correction.

L'import se fait avec `st_read()`, qui lit le shapefile livré dans le package.

```
library(sf)
comtes <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
```

On consulte ensuite le système de coordonnées.

```
st_crs(comtes)
```

```
## Coordinate Reference System:
##   User input: NAD27
##   wkt:
## GEOGCRS["NAD27",
##     DATUM["North American Datum 1927",
##         ELLIPSOID["Clarke 1866",6378206.4,294.978698213898,
##             LENGTHUNIT["metre",1]]],
##     PRIMEM["Greenwich",0,
##         ANGLEUNIT["degree",0.0174532925199433]],
##     CS[ellipsoidal,2],
##         AXIS["latitude",north,
##             ORDER[1],
##             ANGLEUNIT["degree",0.0174532925199433]],
##         AXIS["longitude",east,
##             ORDER[2],
##             ANGLEUNIT["degree",0.0174532925199433]],
##     ID["EPSG",4267]]
```

La couche est dans le système NAD27 (North American Datum 1927), de code EPSG 4267, un système géographique en degrés. On isole maintenant la table attributaire, c'est-à-dire les données sans la géométrie.

```
head(st_drop_geometry(comtes))
```

```
##    AREA PERIMETER CNTY_ CNTY_ID        NAME  FIPS FIPSNO CRESS_ID BIR74 SID74
## 1 0.114     1.442  1825    1825        Ashe 37009  37009        5  1091     1
## 2 0.061     1.231  1827    1827   Alleghany 37005  37005        3   487     0
## 3 0.143     1.630  1828    1828       Surry 37171  37171       86  3188     5
## 4 0.070     2.968  1831    1831   Currituck 37053  37053       27   508     1
## 5 0.153     2.206  1832    1832 Northampton 37131  37131       66  1421     9
## 6 0.097     1.670  1833    1833    Hertford 37091  37091       46  1452     7
##   NWBIR74 BIR79 SID79 NWBIR79
## 1      10  1364     0      19
## 2      10   542     3      12
## 3     208  3616     6     260
## 4     123   830     2     145
## 5    1066  1606     3    1197
## 6     954  1838     5    1237
```

On obtient un tableau ordinaire, sur lequel on peut travailler comme sur n'importe quel data.frame. Enfin, une première carte donne à voir la couche.

```
plot(st_geometry(comtes))
```

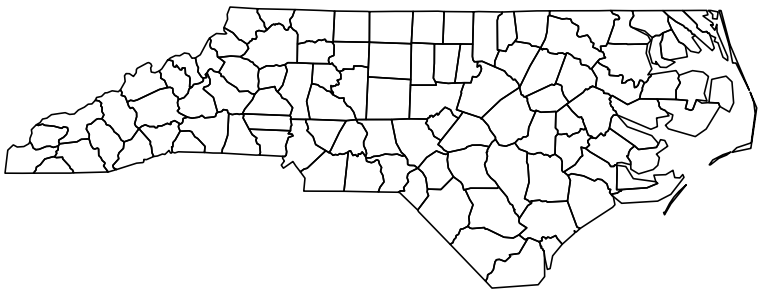


Figure 1: Contours des comtés de Caroline du Nord.

L'usage de `st_geometry()` dans `plot()` n'affiche que les contours, sans colorer les comtés selon une variable, ce qui convient pour une simple prise de vue.

Application 2. Toujours sur les comtés de Caroline du Nord, on veut illustrer les opérations géométriques courantes.

1. Reprojetez la couche dans un système en mètres avec `st_transform()` ;
2. calculez la surface de chaque comté avec `st_area()` ;
3. extrayez le centre de chaque comté avec `st_centroid()` ;
4. tracez une zone tampon autour du premier comté avec `st_buffer()`.

Correction.

Les mesures de surface et de distance exigent un système en mètres. On reprojette donc la couche dans la projection UTM adaptée à la zone (code 32617).

```
comtes_m <- st_transform(comtes, crs = 32617)
```

On calcule alors la surface de chaque comté.

```
comtes_m$surface <- st_area(comtes_m)
```

La colonne obtenue est exprimée en mètres carrés, avec son unité explicitement attachée. On extrait ensuite le centre de chaque comté.

```
centres <- st_centroid(comtes_m)
```

Chaque comté est désormais résumé par un point. À cette étape, R affiche un avertissement signalant qu'il suppose les attributs constants sur chaque comté lors du calcul des centres ; c'est sans incidence sur le résultat. Enfin, on trace une zone tampon de dix kilomètres autour du premier comté.

```
tampon <- st_buffer(comtes_m[1, ], dist = 10000)
plot(st_geometry(tampon))
plot(st_geometry(comtes_m[1, ]), add = TRUE, col = "grey")
```

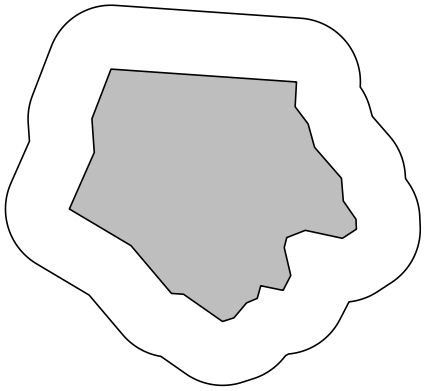


Figure 2: Zone tampon de 10 km autour du premier comté.

La distance est donnée en mètres, conformément au système de la couche reprojetée, d'où la valeur 10000 pour dix kilomètres.

Chapitre 3 : Cartographier, ggplot2, tmap, leaflet

Une fois les données importées et préparées, vient l'étape qui donne à voir : la carte. Trois packages se partagent le terrain, en deux familles. **ggplot2** et **tmap** produisent des cartes statiques, destinées à un rapport ou une publication. **leaflet** produit des cartes interactives, que l'on explore dans un navigateur en zoomant et en cliquant.

Ces trois packages fonctionnent tous par empilement de couches : on part d'un fond, puis on ajoute des éléments par-dessus. C'est une logique proche de celle des logiciels de dessin, où l'on superpose des calques. Les exemples de ce chapitre s'appuient sur des jeux de données livrés avec les packages, donc reproductibles sans téléchargement. Les cartes leaflet, interactives par nature, ne s'affichent pas dans un document PDF ; leur code est donc montré sans être exécuté à la compilation, et leur rendu se découvre en exécutant le code dans RStudio.

3.1 Le package ggplot2

3.1.1 Rôle

ggplot2 est le package de référence pour les graphiques dans R, et il sait aussi faire des cartes. Il repose sur une idée simple : on construit un graphique en empilant des couches, reliées par le signe `+`. On part d'un appel à `ggplot()`, puis on ajoute une couche par jeu de données à afficher. Pour les données spatiales, cette couche s'appelle `geom_sf()`, et elle dessine automatiquement une couche sf, qu'elle contienne des points, des lignes ou des polygones.

Son intérêt est double. D'abord, quiconque connaît déjà ggplot2 pour des graphiques classiques retrouve la même grammaire pour les cartes. Ensuite, une carte ggplot2 s'habille et se combine comme n'importe quel autre graphique, ce qui donne un rendu homogène dans un rapport.

3.1.2 Les fonctions clés

3.1.2.1 Afficher une couche : `geom_sf()`

`geom_sf()` est la couche qui affiche un objet sf. On lui indique les données à dessiner, et éventuellement quelle variable fait varier la couleur.

Ses arguments principaux :

- `data` désigne la couche sf à afficher ;
- `aes()`, pour *aesthetics*, relie une variable des données à un élément visuel de la carte ; par exemple `aes(fill = revenu)` colore chaque polygone selon la variable revenu, et `aes(color = taux)` colore des points selon un taux ;
- ce qui est placé hors de `aes()`, comme `fill = "grey"` ou `color = "red"`, applique une couleur fixe à toute la couche.

```
library(ggplot2)
library(sf)
nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
ggplot() +
  geom_sf(data = nc, aes(fill = AREA))
```

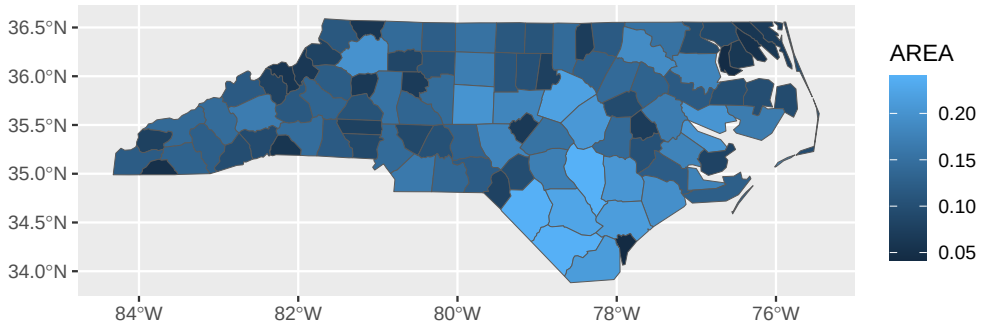


Figure 3: Comtés de Caroline du Nord colorés selon leur superficie (ggplot2).

Ici, `ggplot()` ouvre le graphique et `geom_sf()` dessine les comtés de Caroline du Nord en les colorant selon leur superficie. La distinction est essentielle : une variable placée dans `aes()` fait varier la couleur d'un objet à l'autre, tandis qu'une couleur fixée hors de `aes()` s'applique uniformément.

Pour superposer plusieurs couches, on ajoute simplement plusieurs `geom_sf()`, par exemple un fond de départements puis des points par-dessus. Chaque couche a ses propres données.

3.1.2.2 Choisir les couleurs : les échelles `scale_fill_*` et `scale_color_*`

Une fois une variable reliée à la couleur, on contrôle le dégradé employé avec une fonction d'échelle. `scale_fill_gradient()` définit un dégradé entre deux couleurs pour un remplissage, et `scale_color_gradient()` fait de même pour des points ou des lignes.

Ses arguments courants :

- `low` et `high` fixent les couleurs des valeurs basses et hautes ;
- `name` donne le titre de la légende.

```
ggplot() +
  geom_sf(data = nc, aes(fill = BIR74)) +
  scale_fill_gradient(low = "lightyellow", high = "darkred",
                      name = "Naissances 1974")
```

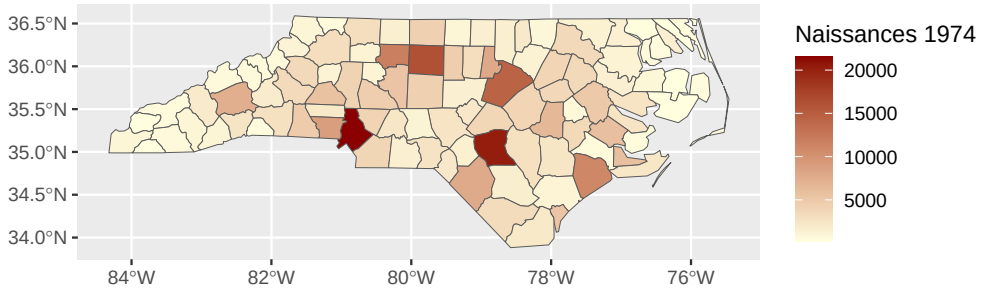


Figure 4: Naissances de 1974 par comté, dégradé jaune-rouge.

Le dégradé du jaune clair au rouge foncé fait ressortir les comtés aux valeurs élevées. Le choix des couleurs n'est pas neutre : un dégradé d'une seule teinte convient à une variable croissante, comme ici.

3.1.2.3 Habiller la carte : `labs()` et les thèmes

`labs()` ajoute les textes autour de la carte, et les fonctions de thème règlent l'apparence générale.

- `labs()` accepte `title`, `subtitle` et `caption` pour le titre, le sous-titre et la source ;
- `theme_void()` retire les axes et le quadrillage, ce qui convient à une carte où les coordonnées n'apportent rien ; `theme_minimal()` garde un fond épuré avec les repères.

```
ggplot() +
  geom_sf(data = nc, aes(fill = AREA)) +
  scale_fill_gradient(low = "white", high = "steelblue", name = "Superficie") +
  labs(title = "Superficie des comtés de Caroline du Nord",
       caption = "Source : jeu de données nc du package sf") +
  theme_void()
```

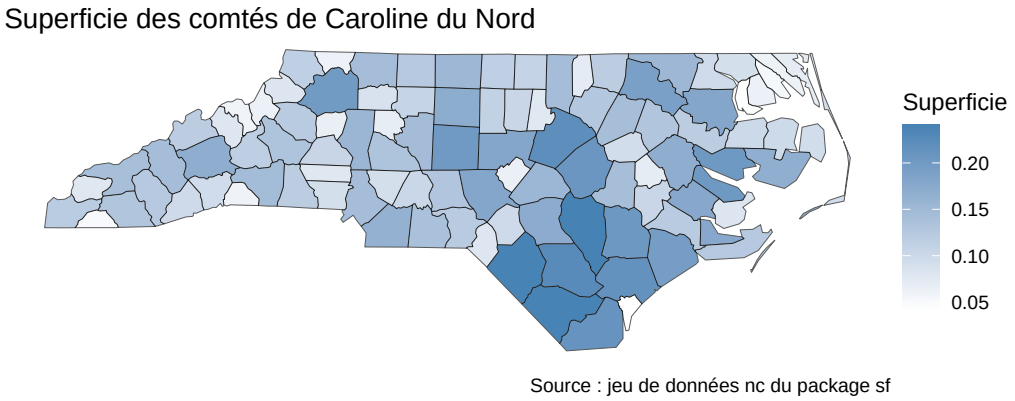


Figure 5: Superficie des comtés avec titre et thème épuré.

Ces fonctions ne changent pas les données représentées, seulement leur présentation. Pour une carte, `theme_void()` est souvent le plus lisible, car la latitude et la longitude en marge n'aident guère la lecture.

3.1.2.4 Les autres fonctions utiles

D'autres fonctions complètent la cartographie avec ggplot2 ; leur code figure en annexe.

`geom_sf_text()` et `geom_sf_label()` ajoutent des étiquettes sur la carte, par exemple le nom de chaque département, placées automatiquement au centre de chaque objet.

`scale_fill_distiller()` et `scale_fill_viridis_c()` offrent des palettes prêtes à l'emploi, plus élaborées qu'un simple dégradé à deux couleurs, utiles pour respecter les règles de lisibilité et d'accessibilité.

`coord_sf()` règle des détails de projection à l'affichage, comme le cadrage ou le système de coordonnées montré en marge.

`facet_wrap()` découpe une carte en plusieurs sous-cartes selon une variable, par exemple une carte par année, toutes tracées sur la même échelle de couleurs. C'est l'outil des comparaisons, à condition d'avoir au préalable réuni les périodes dans une seule colonne avec `pivot_longer()` du tidyverse.

Le tableau suivant récapitule les fonctions présentées :

Usage	Fonction	Rôle
Afficher une couche	`geom_sf()`	dessine une couche sf, points, lignes ou polygones
Colorer selon une variable	`aes(fill=)`, `aes(color=)`	relie une variable à la couleur
Choisir le dégradé	`scale_fill_gradient()`	définit les couleurs de la légende
Étiqueter	`geom_sf_text()`	place le nom des objets sur la carte
Habiller	`labs()`, `theme_void()`	titres, source, apparence générale

3.1.3 Applications

Les exemples s'appuient sur le jeu nc, auquel on ajoute un taux reproductible : le nombre de morts subites du nourrisson pour mille naissances de 1974.

```
library(ggplot2)
nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
nc$taux <- 1000 * nc$SID74 / nc$BIR74
```

Application 1. On veut superposer deux couches : les comtés coloriés selon le taux de mortalité infantile, et le contour extérieur de l'État tracé en trait épais pour bien détacher la région.

1. Calculez le contour de l'État avec `st_union()` ;
2. dessinez les comtés coloriés selon `taux`, puis le contour par-dessus ;
3. choisissez une palette où les valeurs fortes ressortent en foncé.

Correction.

```
contour <- st_union(nc)

ggplot() +
  geom_sf(data = nc, aes(fill = taux), color = "white", linewidth = 0.2) +
  geom_sf(data = contour, fill = NA, color = "black", linewidth = 0.6) +
  scale_fill_distiller(palette = "Reds", direction = 1, name = "Taux") +
  labs(title = "Mortalité infantile en Caroline du Nord, 1974",
       caption = "Source : jeu de données nc du package sf") +
  theme_void()
```

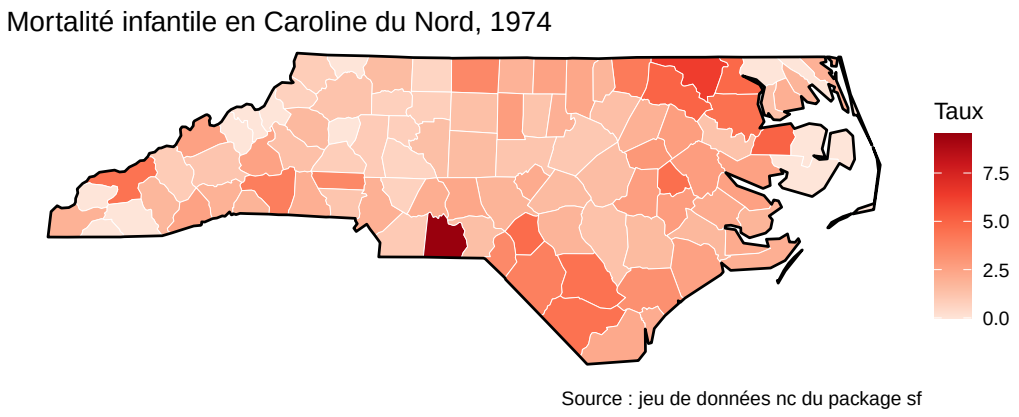


Figure 6: Taux de mortalité infantile et contour de l'État (ggplot2).

Le contour est obtenu avec `st_union()`, vue au chapitre 2, qui fusionne tous les comtés en une seule forme. L'argument `fill = NA` est le point délicat : sans lui, ce grand polygone recouvrirait toute la carte d'une

couleur pleine et cacherait les comtés. Avec `fill = NA`, seul son trait noir subsiste, ce qui encadre proprement la région. La palette `Reds` avec `direction = 1` place le foncé sur les valeurs élevées, comme il se doit pour un phénomène dont on veut voir les points chauds.

Application 2. Les variables SID74 et SID79 donnent les décès pour deux périodes. On veut comparer les taux de 1974 et de 1979 sur deux cartes rigoureusement comparables.

1. Calculez les deux taux ;
2. réunissez-les dans une seule colonne pour pouvoir les cartographier ensemble ;
3. produisez une carte à facettes, une par période.

Correction.

```
library(tidyr)
library(dplyr)
nc_long <- nc |>
  mutate(`1974` = 1000 * SID74 / BIR74,
         `1979` = 1000 * SID79 / BIR79) |>
  pivot_longer(cols = c(`1974`, `1979`),
               names_to = "periode", values_to = "taux_p")

ggplot(nc_long) +
  geom_sf(aes(fill = taux_p), color = "white", linewidth = 0.1) +
  scale_fill_distiller(palette = "Reds", direction = 1, name = "Taux") +
  facet_wrap(~ periode, ncol = 1) +
  theme_void()
```

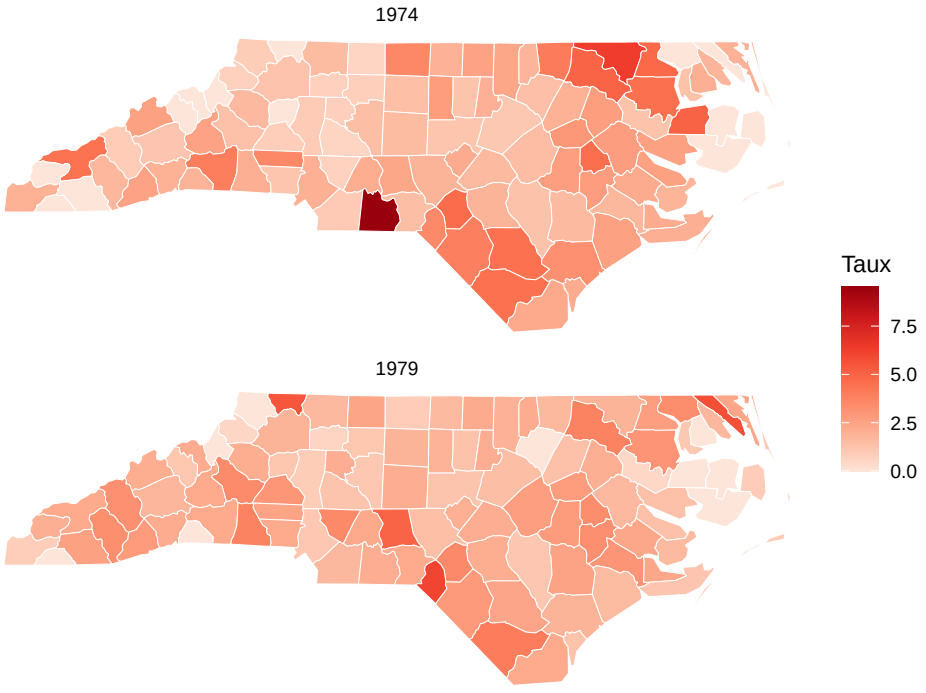


Figure 7: Comparaison des taux de mortalité infantile, 1974 et 1979 (ggplot2).

La manœuvre passe par `pivot_longer()`, qui empile les deux périodes dans une seule colonne : chaque comté existe alors en deux exemplaires, un par année, et `facet_wrap()` produit une carte par période. L'intérêt

est décisif pour la comparaison : les deux cartes partagent automatiquement la même échelle de couleurs, condition sans laquelle on ne pourrait pas comparer honnêtement les deux années, comme le soulignait le chapitre 1.

3.2 Le package tmap

3.2.1 Rôle

tmap est un package dédié à la cartographie thématique. Là où ggplot2 est un outil général adapté aux cartes, tmap est pensé pour elles. Il reprend la même logique de couches reliées par `+`, mais avec un vocabulaire propre à la carte.

Son atout principal est la discrétisation, c'est-à-dire le découpage d'une variable continue en classes de couleurs, qu'il gère finement et selon plusieurs méthodes. Autre force, une même carte tmap peut basculer du mode statique au mode interactif en changeant une seule ligne. Les exemples suivants utilisent la syntaxe de tmap version 4.

3.2.2 Les fonctions clés

3.2.2.1 Poser le fond : `tm_shape()`

Toute carte tmap commence par `tm_shape()`, qui déclare la couche de données à représenter. Seule, cette fonction n'affiche rien : elle prépare le terrain pour les couches visuelles qu'on ajoute ensuite.

- `shp` est la couche sf à cartographier.

```
library(tmap)
tm_shape(nc) +
  tm_polygons()
```

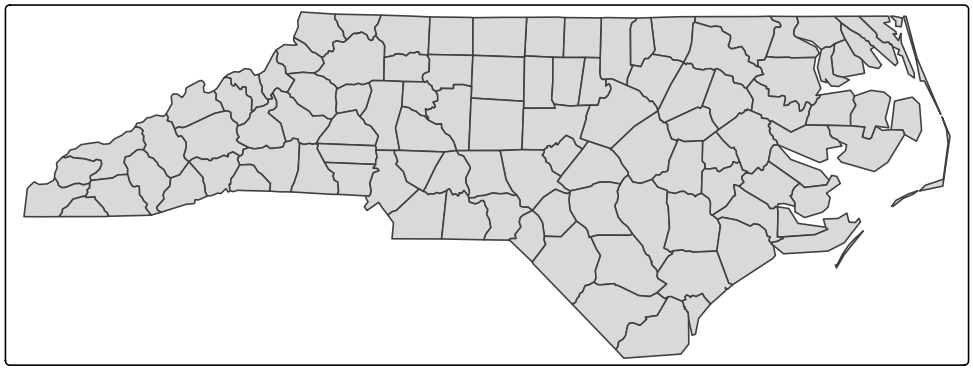


Figure 8: Contours des comtés (tmap).

`tm_shape(nc)` déclare les données, et `tm_polygons()` les dessine. On retrouve la logique de couches : d'abord

on dit quoi représenter, ensuite comment.

3.2.2.2 Dessiner les objets : `tm_polygons()`, `tm_symbols()`, `tm_borders()`

Après `tm_shape()`, une couche visuelle indique comment dessiner les objets, selon leur type :

- `tm_polygons()` remplit et borde des polygones ; c'est la couche la plus courante ;
- `tm_borders()` ne trace que les contours, sans remplissage ;
- `tm_symbols()` place des symboles, utile pour des points ou pour des cercles proportionnels.

Pour colorer selon une variable, on passe le nom de la colonne à l'argument `fill` :

```
tm_shape(nc) +
  tm_polygons(fill = "BIR74")
```

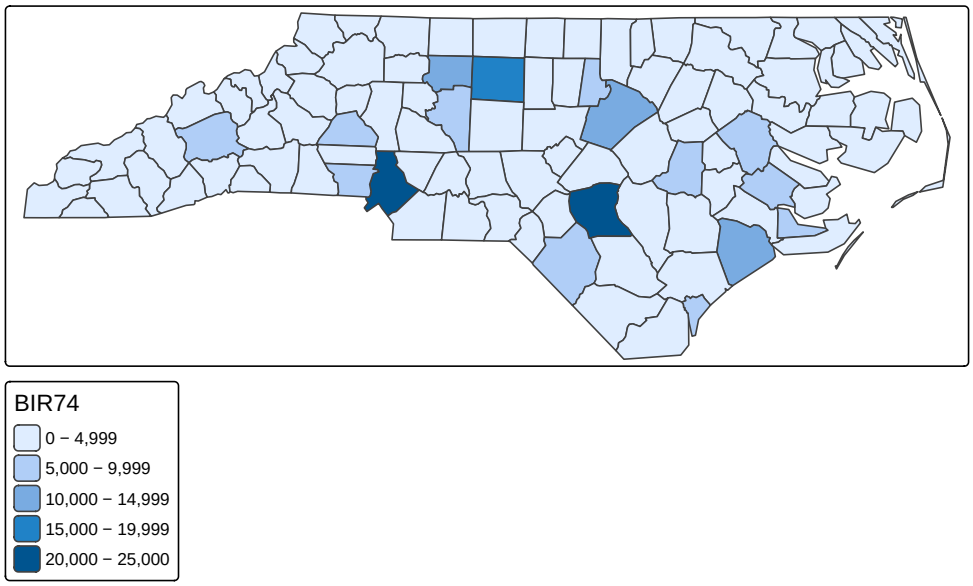


Figure 9: Naissances de 1974, découpage automatique (tmap).

Chaque comté est coloré selon son nombre de naissances. tmap choisit automatiquement un découpage en classes et une palette, qu'on peut ensuite ajuster.

3.2.2.3 Régler les classes et les couleurs : `fill.scale` et `tm_scale()`

C'est ici que tmap se distingue. L'argument `fill.scale`, associé à la fonction `tm_scale()`, contrôle la façon dont la variable est découpée en classes et associée aux couleurs.

- `breaks` fixe des bornes de classes choisies à la main ;
- `n` fixe le nombre de classes en laissant tmap calculer les bornes ;
- `values` choisit la palette de couleurs, par son nom.

À la place de `tm_scale()`, on peut employer `tm_scale_intervals()` pour un découpage en classes selon une méthode précise, par exemple `style = "jenks"` qui forme des classes homogènes.

```r
tm_shape(nc) +
  tm_polygons(fill = "BIR74",
              fill.scale = tm_scale(n = 5, values = "BuGn"))
```

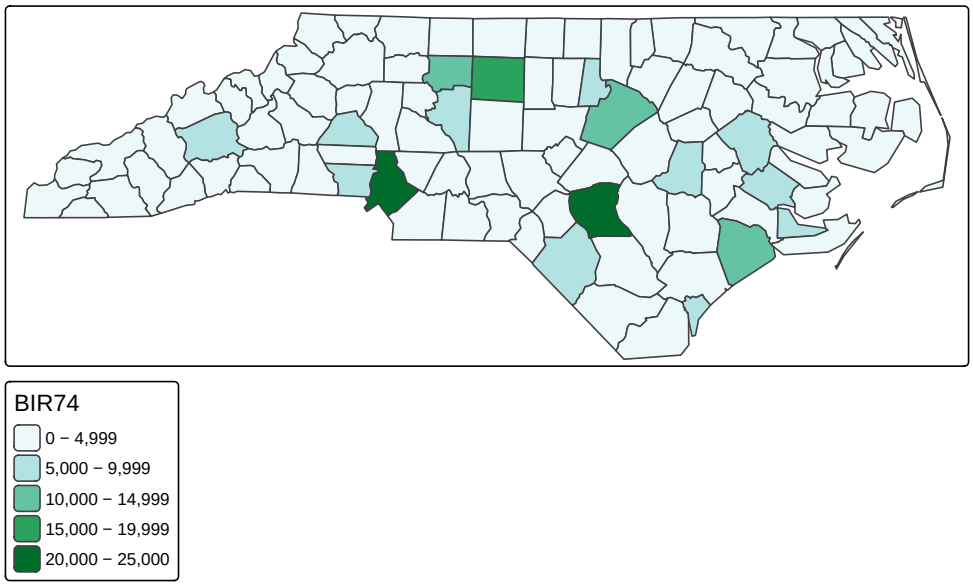


Figure 10: Naissances de 1974 en cinq classes, palette bleu-vert.

Ici la variable est découpée en cinq classes, avec une palette allant du bleu au vert. Le choix du nombre de classes et de la méthode de découpage change la lecture de la carte, comme on l'a vu au chapitre 1 à propos de la discrétisation.

3.2.2.4 Les autres fonctions utiles

D'autres fonctions complètent tmap ; leur code figure en annexe.

L'habillage cartographique se compose de plusieurs fonctions dédiées : `tm_title()` ajoute un titre, `tm_scalebar()` une échelle graphique, `tm_credits()` une mention de source, et `tm_layout()` règle l'apparence générale (marges, légende, fond).

`tm_text()` ajoute des étiquettes textuelles, par exemple le nom des zones.

`tmap_mode()` bascule entre le mode statique (`"plot"`) et le mode interactif (`"view"`). En passant en mode `"view"`, la même carte devient explorable dans le navigateur, avec zoom et fond de carte, sans rien changer d'autre au code. C'est un pont naturel vers leaflet.

Le tableau suivant récapitule les fonctions présentées :

Usage	Fonction	Rôle
Déclarer les données	`tm_shape()`	pose la couche à cartographier
Dessiner	`tm_polygons()`, `tm_symbols()`, `tm_borders()`	remplissage, symboles, contours
Régler classes et couleurs	`fill.scale`, `tm_scale()`, `tm_scale_intervals()`	discrétisation et palette

Usage	Fonction	Rôle
Habiller	`tm_title()`, `tm_scalebar()`, `tm_credits()`, `tm_layout()`	titre, échelle, source, mise en page
Changer de mode	`tmap_mode()`	statique ou interactif

3.2.3 Applications

Application 1. On veut cartographier le taux de mortalité infantile en cinq classes construites par la méthode de Jenks, avec un habillage complet.

1. Déclarez la couche avec `tm_shape()` ;
2. coloriez selon `taux` en cinq classes Jenks avec `fill.scale` ;
3. ajoutez un titre, une échelle et une source.

Correction.

```
library(tmap)
tm_shape(nc) +
  tm_polygons(fill = "taux",
              fill.scale = tm_scale_intervals(style = "jenks", n = 5,
                                              values = "brewer.reds"),
              fill.legend = tm_legend(title = "Taux pour 1 000")) +
  tm_title("Mortalité infantile en Caroline du Nord, 1974") +
  tm_scalebar(position = c("left", "bottom")) +
  tm_credits("Source : jeu de données nc du package sf")
```

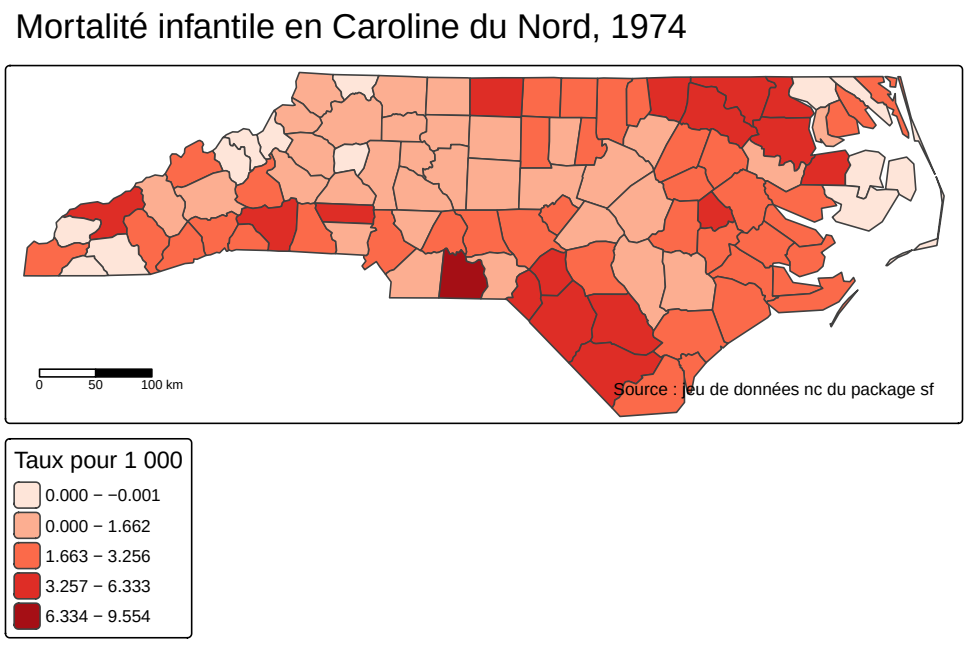


Figure 11: Mortalité infantile en cinq classes Jenks (tmap).

Cinq classes pour cent comtés respectent la règle pratique du chapitre 1. La méthode de Jenks forme des classes homogènes en regroupant les valeurs proches, et la palette de rouges ordonnée place le foncé sur

les taux élevés. L'habillage (titre, échelle, source) fait la différence entre une carte de travail et une carte présentable.

Application 2. La variable BIR74 compte les naissances par comté. On veut montrer pourquoi une choroplèthe serait fautive ici, puis produire la bonne représentation.

1. Expliquez le problème d'une choroplèthe sur un effectif brut ;
2. représentez les naissances en cercles proportionnels avec `tm_symbols()`.

Correction.

```
tm_shape(nc) +
  tm_polygons(fill = "grey95") +
  tm_shape(nc) +
  tm_symbols(size = "BIR74", fill = "steelblue", fill_alpha = 0.7,
             size.legend = tm_legend(title = "Naissances 1974"))
```

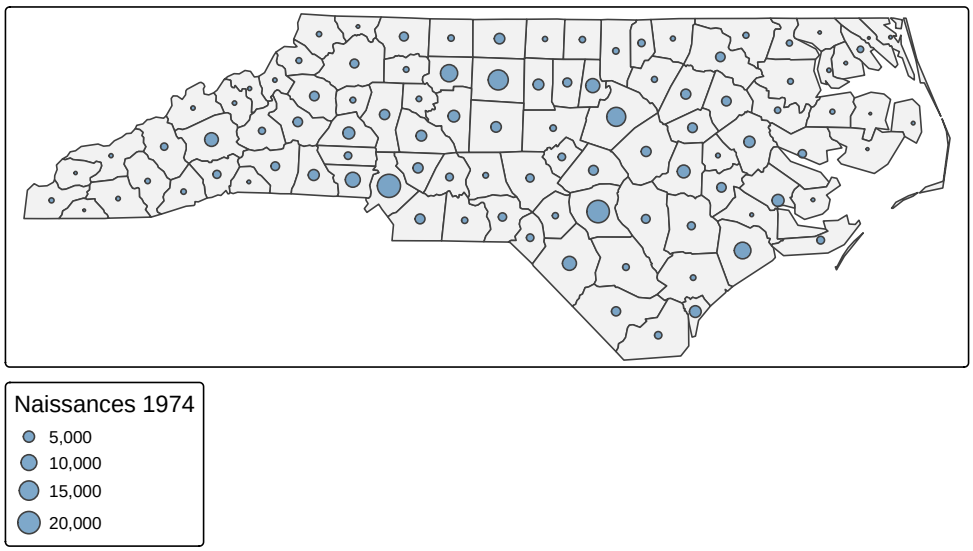


Figure 12: Naissances de 1974 en cercles proportionnels (tmap).

BIR74 est un effectif brut, pas un taux. En aplats de couleurs, un grand comté paraîtrait important du seul fait de sa surface, le piège décrit au chapitre 1. Les cercles proportionnels lèvent cette distorsion : l'œil compare des tailles de cercles, indépendantes de la surface des comtés. On pose d'abord un fond gris neutre avec `tm_polygons`, puis on superpose les cercles avec `tm_symbols`, dont l'argument `size` fait varier le rayon selon la variable.

3.3 Le package leaflet

3.3.1 Rôle

leaflet produit des cartes interactives, celles qu'on explore dans un navigateur en zoomant, en déplaçant la vue et en cliquant sur les objets. Il s'appuie sur la bibliothèque JavaScript du même nom, très répandue sur le web.

Contrairement à ggplot2 et tmap, leaflet n'est pas fait pour le papier : ses cartes vivent à l'écran. Il est donc idéal pour explorer des données ou pour une publication en ligne, mais ses cartes ne s'impriment pas dans un PDF. Comme les précédents, il fonctionne par ajout de couches, avec l'opérateur `%>%` au lieu du `+`.

3.3.2 Les fonctions clés

3.3.2.1 Ouvrir la carte et poser un fond : `leaflet()` et `addTiles()`

`leaflet()` crée la carte vide, et `addTiles()` y ajoute un fond de carte, par défaut celui d'OpenStreetMap.

```
library(leaflet)
leaflet() %>%
  addTiles()
```

On obtient une carte du monde vide, explorable, sur laquelle on ajoutera ensuite ses données. Le fond fournit le contexte géographique, villes, routes, reliefs.

3.3.2.2 Ajouter des objets : `addPolygons()` et `addCircleMarkers()`

On superpose ensuite ses données au fond, selon leur type :

- `addPolygons()` ajoute des polygones, avec des arguments comme `fillColor` pour la couleur de remplissage et `popup` pour le texte affiché au clic ;
- `addCircleMarkers()` ajoute des points sous forme de cercles, utile pour des lieux ou des formations.

```
nc_wgs <- st_transform(nc, crs = 4326)
leaflet(nc_wgs) %>%
  addTiles() %>%
  addPolygons(fillColor = "steelblue", weight = 1,
              popup = ~NAME)
```

Chaque comté devient cliquable et affiche son nom. Un point mérite attention : leaflet attend des coordonnées en WGS 84 (code 4326), le système des GPS ; il faut donc reprojeter une couche qui serait en Lambert 93 avant de l'afficher, avec `st_transform()`.

3.3.2.3 Les autres fonctions utiles

D'autres fonctions complètent leaflet ; leur code figure en annexe.

`addProviderTiles()` change le fond de carte pour un autre fournisseur, par exemple un fond satellite ou un fond épuré.

`addLegend()` ajoute une légende, indispensable dès qu'une couleur code une variable.

`colorNumeric()`, `colorBin()` et `colorFactor()` construisent une palette reliant les valeurs aux couleurs, à passer ensuite à `addPolygons()` ou `addCircleMarkers()` ; `colorBin()` découpe en classes, `colorNumeric()` fait un dégradé continu.

Le tableau suivant récapitule les fonctions présentées :

Usage	Fonction	Rôle
Ouvrir la carte	`leaflet()`	crée la carte interactive
Poser un fond	`addTiles()`, `addProviderTiles()`	fond OpenStreetMap ou autre
Ajouter des objets	`addPolygons()`, `addCircleMarkers()`	polygones et points cliquables
Habiller	`addLegend()`, `colorNumeric()`	légende et palettes

3.3.3 Applications

Application 1. On veut une carte interactive des comtés, coloriés selon le taux de mortalité infantile, chaque comté affichant son nom et sa valeur au clic, avec une légende.

1. Reprojetez la couche en WGS 84 avec `st_transform()` ;
2. construisez une palette de couleurs par classes avec `colorBin()` ;
3. ajoutez les comtés coloriés, cliquables, et une légende.

Correction.

```
library(leaflet)
nc_wgs <- st_transform(nc, 4326)
pal <- colorBin("Reds", domain = nc_wgs$taux, bins = 5)

leaflet(nc_wgs) %>%
  addTiles() %>%
  addPolygons(fillColor = ~pal(taux), fillOpacity = 0.7,
              color = "white", weight = 1,
              popup = ~paste0(NAME, " : ", round(taux, 1), " pour 1 000")) %>%
  addLegend(pal = pal, values = ~taux, title = "Taux pour 1 000")
```

`colorBin()` construit une palette qui découpe le taux en cinq classes de rouge, comme le ferait tmap, mais utilisable par leaflet. Chaque comté devient cliquable et affiche son nom et sa valeur. La carte dit la même chose que sa version statique, mais le lecteur interroge chaque comté au lieu de deviner sa valeur dans la légende. Pour un rapport PDF on garde la version statique ; l'interactive prend son sens sur une page web ou en réunion.

Application 2. On veut afficher des points cliquables : le centre de chaque comté, sous forme de cercle, avec le nom du comté au clic.

1. Calculez les centres avec `st_centroid()` et reprojetez-les en WGS 84 ;
2. affichez-les avec `addCircleMarkers()`, cliquables sur le nom.

Correction.

```
centres <- st_centroid(nc)
centres_wgs <- st_transform(centres, 4326)

leaflet(centres_wgs) %>%
  addTiles() %>%
  addCircleMarkers(radius = 5, color = "darkred", fillOpacity = 0.8,
                   popup = ~paste("Comté :", NAME))
```

`addCircleMarkers()` place un cercle par point, ici le centre de chaque comté obtenu avec `st_centroid()` du chapitre 2. Comme pour les polygones, leaflet exige des coordonnées en WGS 84, d'où la reprojection. Le rendu interactif ne peut pas figurer dans ce document imprimé ; il faut exécuter le code dans RStudio pour l'explorer.

Chapitre 4 : Traiter les données raster, terra et stars

Le chapitre 1 a distingué deux façons de représenter l'espace : le modèle vectoriel, où chaque objet est dessiné par ses coordonnées, et le modèle raster, où le territoire est découpé en une grille régulière de cellules portant chacune une valeur. Tous les exemples vus jusqu'ici relevaient du vectoriel ; ce chapitre est consacré au raster.

Deux mots de vocabulaire suffisent pour la suite. Une cellule de la grille s'appelle couramment un **pixel**, exactement comme dans une photo numérique : en zoomant très fort sur une image à l'écran, on finit par voir ces petits carrés, et un raster n'est rien d'autre que cela, des carrés porteurs de valeurs. Par ailleurs, un même fichier raster peut contenir plusieurs grilles superposées sur le même territoire ; chacune s'appelle alors une **bande**. Une photo satellite en couleurs en contient trois, une pour le rouge, une pour le vert, une pour le bleu ; une série de relevés de température en contient une par date de mesure.

Deux packages permettent de traiter des rasters dans R. **terra** est l'outil généraliste ; il remplace progressivement l'ancien package raster, écrit par le même auteur. **stars** est plus spécialisé : il représente les rasters sous forme de cubes de données et fonctionne directement avec les objets sf. Comme dans les chapitres précédents, la plupart des exemples reposent sur des fichiers fournis avec les packages, ici l'altitude du Luxembourg et la couche de ses douze cantons, reproductibles sans téléchargement. Une application utilise en complément le package spDataLarge, à installer séparément. Plusieurs exercices s'inspirent par ailleurs d'exemples de la documentation et de la littérature sur R spatial.

4.1 Le package terra : le successeur de raster

4.1.1 Rôle

Pendant une dizaine d'années, traiter un raster dans R passait par le package raster. Celui-ci a été publié en 2010 par Robert Hijmans et est resté la référence jusqu'à l'arrivée de terra et de stars (Lovelace et al., 2025). En 2020, Hijmans a publié terra pour remplacer son propre package. L'interface reste très proche, mais le code est réécrit en C++ et nettement plus rapide. La transition est moins brutale que celle de sp vers sf vue au chapitre 2. La plupart des fonctions gardent le même nom et la même logique. Seules les classes d'objets changent, et la conversion des anciens objets vers les nouveaux reste possible.

terra repose sur deux classes principales. `SpatRaster` représente les rasters, `SpatVector` les données vectorielles. Cette seconde classe surprend au début : terra possède son propre format vectoriel, différent de sf. Ce n'est pas un problème en pratique. Beaucoup de fonctions acceptent directement les objets sf, et `vect()` fait la conversion quand une opération l'exige, par exemple pour découper un raster avec un polygone.

Dernier point, décisif sur les gros fichiers : terra ne charge pas forcément toutes les valeurs en mémoire. À l'ouverture, il lit l'en-tête et garde un simple lien vers le fichier sur le disque. Pour les calculs, il peut traiter l'image par morceaux au lieu de tout monter dans la RAM (Lovelace et al., 2025). On peut ainsi manipuler des images de plusieurs gigaoctets sur un ordinateur ordinaire. Les erreurs de mémoire surviennent surtout quand on force le chargement complet, par exemple avec `as.data.frame()` ou `values()`. Le bon réflexe est alors de découper ou d'agréger l'image avant la conversion.

4.1.2 Les fonctions clés

4.1.2.1 Importer et inspecter : `rast()`

Pour charger une image géoréférencée dans R, la fonction d'entrée est `rast()`. Elle lit la plupart des formats raster (GeoTIFF en tête, mais aussi les formats ESRI, ENVI ou ERDAS) et renvoie un objet de classe SpatRaster.

Son usage le plus courant ne demande qu'un seul argument :

- `x` est la source du raster, le plus souvent le chemin vers le fichier ; ce peut aussi être un objet existant à convertir, ou les dimensions d'une grille à créer de toutes pièces.

```
library(terra)
alt <- rast(system.file("ex/elev.tif", package = "terra"))
alt
```

```
## class       : SpatRaster
## size        : 90, 95, 1  (nrow, ncol, nlyr)
## resolution  : 0.008333333, 0.008333333  (x, y)
## extent      : 5.741667, 6.533333, 49.44167, 50.19167  (xmin, xmax, ymin, ymax)
## coord. ref. : lon/lat WGS 84 (EPSG:4326)
## source      : elev.tif
## name        : elevation
## min value   :       141
## max value   :       547
```

L'affichage donne la carte d'identité du raster : ses dimensions, sa résolution, son emprise, son système de coordonnées et les valeurs extrêmes. Quelques fonctions livrent ces informations une à une :

- `res()` donne la résolution, c'est-à-dire la taille d'un pixel ;
- `ext()` donne l'emprise, le rectangle couvert par l'image ;
- `nlyr()` donne le nombre de bandes ;
- `ncell()` donne le nombre de pixels.

```
plot(alt, main = "Altitude du Luxembourg (m)")
```

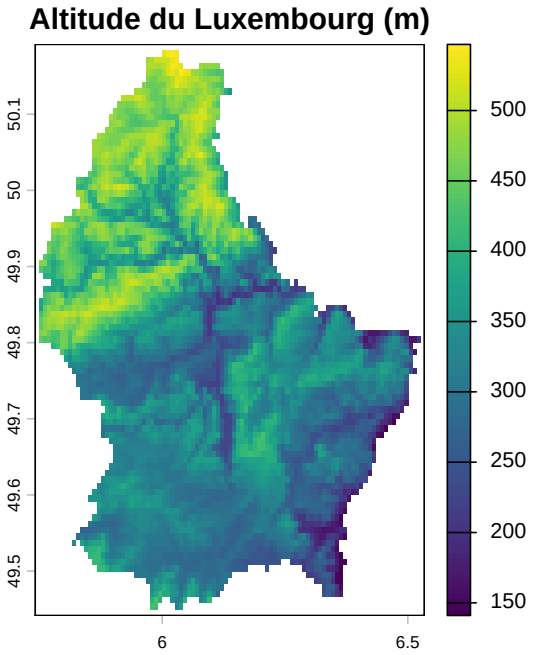


Figure 13: Altitude du Luxembourg, en mètres.

4.1.2.2 Explorer les valeurs : `summary()`, `global()` et `freq()`

Avant de traiter un raster, il faut regarder ce qu'il contient. Taper le nom de l'objet affiche déjà les valeurs minimale et maximale. Trois fonctions vont plus loin :

- `summary()` donne les quartiles et le nombre de valeurs manquantes sur un raster continu, ou le nombre de pixels par classe sur un raster catégoriel ;

- `global()` calcule une statistique précise sur toute l'image, par exemple la moyenne ou l'écart-type ;
- `freq()` compte les pixels de chaque classe, utile sur un raster catégoriel.

```
global(alt, "mean", na.rm = TRUE)
```

```
##                mean
## elevation 348.3366
```

L'argument `na.rm = TRUE` est presque toujours nécessaire : les pixels situés hors du territoire portent la valeur NA, et sans cet argument le résultat serait NA. Sur un raster à plusieurs bandes, ces fonctions traitent chaque bande séparément. Le code détaillé de `summary()`, `freq()` et des fonctions graphiques comme `hist()` figure en annexe.

Une mise en garde utile : certains noms de fonctions existent dans plusieurs packages. `extract()`, par exemple, existe chez terra et chez tidyr, et selon l'ordre de chargement, R peut appeler la mauvaise. En cas de doute, on écrit le nom complet, `terra::extract()`.

4.1.2.3 Calculer sur les pixels : l'algèbre de cartes et `classify()`

Un raster est au fond un grand tableau de nombres. On peut donc faire des calculs dessus, et ils s'appliquent à tous les pixels d'un coup. Cette façon de calculer s'appelle l'**algèbre de cartes**.

Ces calculs se classent en quatre familles, selon l'information utilisée pour produire chaque résultat (Tomlin, 1990, cité par Lovelace et al., 2025). Une opération est dite **locale** quand chaque pixel est calculé à partir de sa seule valeur. Elle est **focale** quand le calcul fait aussi intervenir les pixels voisins, par exemple pour adoucir une image. Elle est **zonale** quand on résume les pixels zone par zone, comme le fera `extract()` pour donner l'altitude moyenne de chaque canton. Elle est **globale** quand le calcul porte sur l'image entière, comme la moyenne renvoyée par `global()`. Ce chapitre illustre trois de ces familles ; les opérations focales, plus spécialisées, n'y figurent pas.

L'opération locale la plus utile en pratique est la reclassification : transformer une variable continue en classes. C'est le rôle de `classify()`, qui repose sur deux arguments :

- `rcl` est la matrice de correspondance, le plus souvent à trois colonnes : la borne basse, la borne haute, et le code attribué à cette tranche de valeurs ;
- `include.lowest`, mis à `TRUE`, garantit que le pixel le plus bas est bien rangé dans la première classe au lieu d'être laissé de côté.

```
bornes <- matrix(c(0, 300, 1,
                   300, 450, 2,
                   450, 600, 3), ncol = 3, byrow = TRUE)
alt_classes <- classify(alt, bornes)
plot(alt_classes, main = "Altitude en trois classes")
```

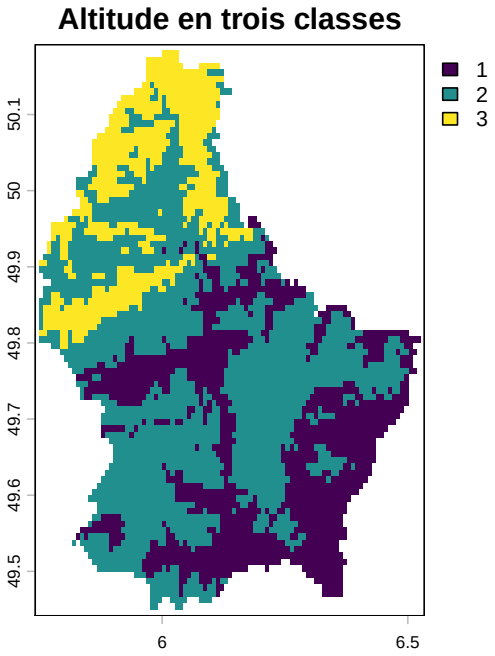


Figure 14: Altitude reclassée en trois classes.

La matrice se lit ligne par ligne : de 0 à 300 mètres, classe 1 ; de 300 à 450, classe 2 ; au-delà, classe 3. Le raster obtenu est catégoriel.

4.1.2.4 Découper : `crop()` et `mask()`

Un raster couvre souvent une zone plus large que la zone d'étude. On le découpe alors, pour ne garder que la zone utile et alléger les calculs (Lovelace et al., 2025). Deux fonctions s'en chargent, et on les confond facilement :

- `crop(x, y)` rogne l'image x au rectangle qui englobe y ; le résultat reste rectangulaire ;
- `mask(x, y)` garde les dimensions de x mais met à NA tous les pixels situés hors de y ; le résultat épouse la forme du polygone.

En pratique, on enchaîne souvent les deux : `crop()` pour réduire la taille du fichier, puis `mask()` pour obtenir la forme exacte.

```
lux <- vect(system.file("ex/lux.shp", package = "terra"))
diekirch <- lux[lux$NAME_1 == "Diekirch", ]
alt_diekirch <- mask(crop(alt, diekirch), diekirch)
plot(alt_diekirch, main = "Altitude du district de Diekirch")
```

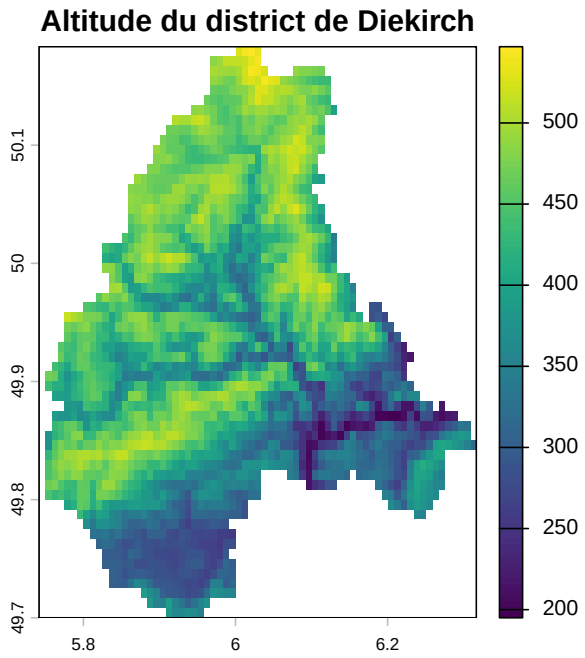


Figure 15: Altitude du district de Diekirch après découpage.

Deux précautions avant de découper. Les deux couches doivent partager le même système de coordonnées, sinon le découpage tombe à côté ou échoue. Et la couche de découpe doit être un objet terra : c'est le rôle de `vect()`, qui accepte un chemin de fichier comme ici, ou un objet sf à convertir.

4.1.2.5 Croiser raster et vecteur : `extract()`

`extract()` répond à une question très courante : que vaut le raster à tel endroit ? On lui donne le raster et une couche vectorielle, et elle va chercher les valeurs correspondantes (Lovelace et al., 2025). Le résultat dépend du type de couche :

- sur des points, chaque point reçoit la valeur du pixel où il tombe ;
- sur des polygones, chaque polygone couvre de nombreux pixels, et l'argument `fun` permet de les résumer par une statistique.

```
alt_moyenne <- extract(alt, lux, fun = mean, na.rm = TRUE)
lux$alt_moy <- alt_moyenne[, 2]
head(as.data.frame(lux[, c("NAME_2", "alt_moy")]), 4)
```

```
##     NAME_2  alt_moy
## 1 Clervaux 467.1052
## 2 Diekirch 333.8629
## 3  Redange 377.3712
## 4  Vianden 373.6000
```

Chaque canton reçoit ici son altitude moyenne. La fonction renvoie un data.frame dont la première colonne numérote les polygones et la seconde porte la statistique, d'où le `[, 2]` pour la récupérer.

4.1.2.6 Changer de résolution et de projection : `aggregate()` et `project()`

Deux fonctions complètent la chaîne de traitement ; leur code figure en annexe.

`aggregate()` réduit la résolution d'un raster en regroupant les pixels. L'argument `fact` fixe la taille des blocs et `fun` la façon de les résumer : avec `fact = 4`, chaque bloc de quatre pixels sur quatre devient un seul pixel, et l'image compte seize fois moins de cellules. C'est la parade la plus simple aux problèmes de mémoire sur les gros fichiers.

`project()` reprojette un raster, comme `st_transform()` le fait pour un vecteur. Mais l'opération coûte plus cher : pour un vecteur, on recalcule les coordonnées de chaque sommet ; pour un raster, la grille devant rester régulière, il faut construire une nouvelle grille dans le système cible puis estimer la valeur de chaque pixel. Chaque reprojection altère donc un peu les valeurs, d'où la règle pratique : quand on combine rasters et vecteurs, on reprojette les vecteurs vers le système du raster, et pas l'inverse.

4.1.3 Applications

Application 1. Un organisme vous transmet le fichier d'altitude elev.tif sans documentation. À l'aide du package terra :

1. importez le raster avec `rast()` ;
2. donnez sa résolution, son nombre de pixels et son système de coordonnées ;
3. déterminez l'altitude maximale du territoire couvert.

Correction.

L'import se fait avec `rast()`, qui lit le fichier livré dans le package.

```
library(terra)
alt <- rast(system.file("ex/elev.tif", package = "terra"))
```

Les fonctions d'inspection renvoient ensuite les caractéristiques demandées.

```
res(alt)                          # résolution
```

```
## [1] 0.008333333 0.008333333
```

```
ncell(alt)                        # nombre de pixels
```

```
## [1] 8550
```

```
crs(alt, describe = TRUE)$name    # système de coordonnées
```

```
## [1] "WGS 84"
```

La résolution s'exprime en degrés, puisque le raster est en coordonnées géographiques, et l'image compte 8 550 pixels. Enfin, l'altitude maximale s'obtient avec `global()`.

```
global(alt, "max", na.rm = TRUE)
```

```
##           max
## elevation 547
```

Le point culminant approche les 550 mètres. Petit piège : sans `na.rm = TRUE`, le maximum renvoyé serait NA, à cause des pixels vides autour du pays.

Application 2. On souhaite décrire la répartition du relief luxembourgeois.

1. À l'aide de `classify()`, reclassez les altitudes en trois classes : basses, moyennes et hautes ;
2. cartographiez le résultat ;
3. avec `freq()`, comptez le nombre de pixels de chaque classe et commentez leur répartition.

Correction.

On reclasse les altitudes en trois classes avec `classify()`, qui attend une matrice à trois colonnes : la borne basse, la borne haute et le code de la classe (Lovelace et al., 2025). Plutôt que de fixer les bornes au jugé, on les place aux quantiles à un tiers et deux tiers de la distribution : chaque classe contient ainsi environ un tiers du territoire, au principe vu au chapitre 1.

```
seuils <- quantile(values(alt, na.rm = TRUE), probs = c(1/3, 2/3))
rcl <- matrix(c(141, seuils[1], 1,
                seuils[1], seuils[2], 2,
                seuils[2], 547, 3), ncol = 3, byrow = TRUE)
alt_cl <- classify(alt, rcl, include.lowest = TRUE)
levels(alt_cl) <- data.frame(id = 1:3,
                             relief = c("Basses", "Moyennes", "Hautes"))
```

`values()` extrait les altitudes, `quantile()` en tire les deux bornes intérieures, et la matrice les organise en trois classes. L'argument `include.lowest = TRUE` évite que le pixel le plus bas soit exclu, et `levels()` remplace les codes numériques par des étiquettes lisibles. On cartographie le résultat.

```
plot(alt_cl, main = "Altitudes du Luxembourg en trois classes")
```

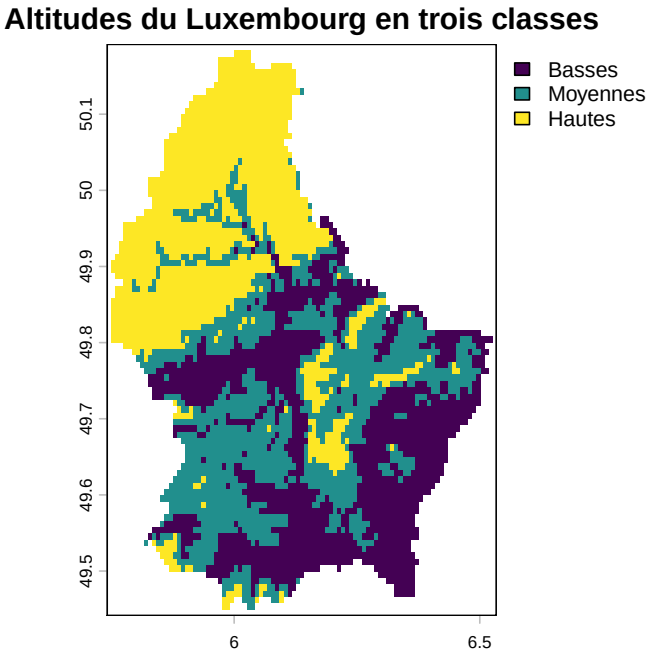


Figure 16: Altitudes du Luxembourg réparties en trois classes.

Reste à compter les pixels de chaque classe.

```
freq(alt_cl)
```

```
##   layer    value count
## 1     1   Basses  1558
## 2     1 Moyennes  1521
## 3     1   Hautes  1529
```

Les trois classes comptent chacune environ 1500 pixels, ce qui est attendu puisque les quantiles découpent le territoire en tiers. Sur la carte, les altitudes basses occupent le sud du pays et les hautes le nord.

Application 3. On veut illustrer la différence entre `crop()` et `mask()` sur une zone aux contours irréguliers. On utilise pour cela un modèle d'altitude du parc de Zion et un nuage de points, tous deux fournis par le package spDataLarge (contrairement aux autres applications, celle-ci nécessite donc ce téléchargement).

1. Construisez une zone de découpe aux contours irréguliers à partir des points ;
2. découpez le raster sur cette zone avec `crop()` seul, puis avec `crop()` suivi de `mask()` ;
3. affichez les deux résultats côte à côte et expliquez ce qui les distingue.

Correction.

On charge les deux jeux de données, puis on fabrique une zone de découpe à partir des points. Comme un nuage de points ne peut pas servir directement à découper un raster, il faut d'abord en tirer une surface. C'est le rôle des deux fonctions suivantes : `st_combine()` rassemble les points épars en un seul objet géométrique, et `st_convex_hull()` trace autour d'eux leur enveloppe convexe, c'est-à-dire le plus petit polygone qui les contient tous, comme un élastique tendu autour. On obtient ainsi un polygone à la forme oblique, qui ne remplit pas son rectangle englobant : c'est exactement ce qu'il faut pour bien voir la différence entre crop et mask.

```
library(sf)
library(spDataLarge)
srtm <- rast(system.file("raster/srtm.tif", package = "spDataLarge"))
points_zion <- read_sf(system.file("vector/zion_points.gpkg", package = "spDataLarge"))
enveloppe <- st_as_sf(st_convex_hull(st_combine(points_zion)))
```

On réalise ensuite les deux découpages. On fait précéder `mask()` d'un `crop()` pour ajuster l'emprise : `mask()` seul donnerait la même forme, mais conserverait le grand rectangle du raster d'origine, avec beaucoup de vide autour de la zone utile. `crop()` rétrécit d'abord le rectangle aux dimensions de l'enveloppe, puis `mask()` en vide les coins.

```
srtm_crop <- crop(srtm, vect(enveloppe))
srtm_mask <- mask(srtm_crop, vect(enveloppe))
par(mfrow = c(1, 2))
plot(srtm_crop, main = "crop seul")
plot(srtm_mask, main = "crop puis mask")
```

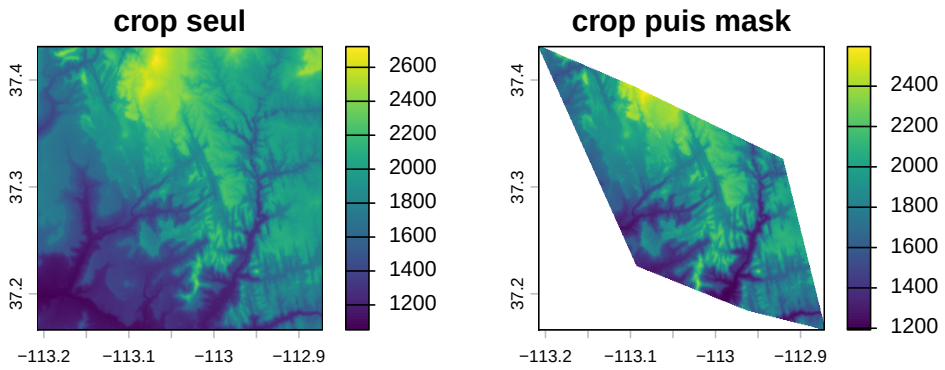


Figure 17: Découpage par crop seul (gauche) et par crop puis mask (droite).

```
par(mfrow = c(1, 1))
```

La différence est nette. `crop()` réduit le raster au rectangle qui entoure l'enveloppe : l'image reste rectangulaire et entièrement remplie. `mask()` met ensuite à NA tous les pixels situés hors de l'enveloppe : il vide les coins du rectangle, et seul le polygone oblique subsiste. Pour produire une carte épousant la forme exacte d'un territoire, on utilise donc mask ; pour seulement réduire la taille d'un fichier avant un calcul, crop suffit.

Application 4. On veut déterminer quel canton du Luxembourg culmine en moyenne le plus haut.

1. À l'aide d'`extract()` et de la couche des cantons fournie avec terra, calculez l'altitude moyenne de chacun des douze cantons ;
2. identifiez le canton le plus élevé.

Correction.

L'altitude moyenne de chaque canton s'obtient avec `extract()` et l'argument `fun`.

```
moy <- extract(alt, lux, fun = mean, na.rm = TRUE)
lux$alt_moy <- moy[, 2]
```

On identifie ensuite le canton le plus élevé.

```
resultat <- as.data.frame(lux[, c("NAME_2", "alt_moy")])
resultat[which.max(resultat$alt_moy), ]
```

```
##     NAME_2  alt_moy
## 1 Clervaux 467.1052
```

Le canton le plus haut en moyenne est Clervaux, au nord du pays, avec une altitude moyenne d'environ 467 mètres. Cette opération, transformer une grille continue en une variable par zone, est exactement celle qu'on réaliserait pour attribuer à chaque commune normande une altitude ou une donnée climatique moyenne.

4.2 Le package stars : raster et cubes de données spatio-temporels

4.2.1 Rôle

terra suffit pour la plupart des rasters, mais il atteint ses limites quand les données prennent plus de deux dimensions. C'est là qu'intervient stars, développé par Edzer Pebesma dans le prolongement de sf.

L'idée centrale est celle de **cube de données**. Une image n'a pas toujours que deux dimensions spatiales. Une photo satellite a aussi des bandes spectrales, et une série d'images ajoute le temps. On obtient un tableau à plusieurs entrées : longitude, latitude, bande, date. Chaque case contient une valeur. C'est ce tableau multidimensionnel qu'on appelle un cube, même quand il a plus de trois dimensions.

stars représente ces cubes directement, là où terra empile des couches. Il partage les conventions de sf : ses fonctions commencent par `st_` et il s'intègre au tidyverse. Pour un raster simple, terra reste le plus pratique ; stars prend l'avantage dès qu'intervient le temps, ou quand on veut rester dans l'écosystème sf.

Les exemples utilisent l'image Landsat 7 fournie avec le package, six bandes spectrales sur une zone de la côte brésilienne.

4.2.2 Les fonctions clés

4.2.2.1 Importer et décrire : `read_stars()` et `st_dimensions()`

Pour charger un cube de données dans R, la fonction d'entrée est `read_stars()`. Comme `rast()` chez terra, elle lit une image géoréférencée, mais elle la range sous forme de cube, c'est-à-dire un tableau à plusieurs dimensions.

Son usage courant ne demande qu'un argument :

- `.x` est la source des données, le plus souvent le chemin vers le fichier.

```
library(stars)
l7 <- read_stars(system.file("tif/L7_ETMs.tif", package = "stars"))
l7
```

```
## stars object with 3 dimensions and 1 attribute
## attribute(s):
##              Min. 1st Qu. Median     Mean 3rd Qu. Max.
## L7_ETMs.tif     1      54     69 68.91242      86  255
## dimension(s):
##      from  to  offset delta                       refsys point x/y
## x       1 349  288776  28.5 SIRGAS 2000 / UTM zone 25S FALSE [x]
## y       1 352 9120761 -28.5 SIRGAS 2000 / UTM zone 25S FALSE [y]
## band    1   6      NA    NA                           NA    NA
```

L'affichage diffère de celui de terra : il décrit l'objet par ses dimensions x, y et band, avec pour chacune son nombre de pas. La fonction `st_dimensions(l7)` donne ce détail seul. On lit alors l'objet comme sa définition : un cube de 349 sur 352 pixels, sur 6 bandes.

```
plot(l7)
```

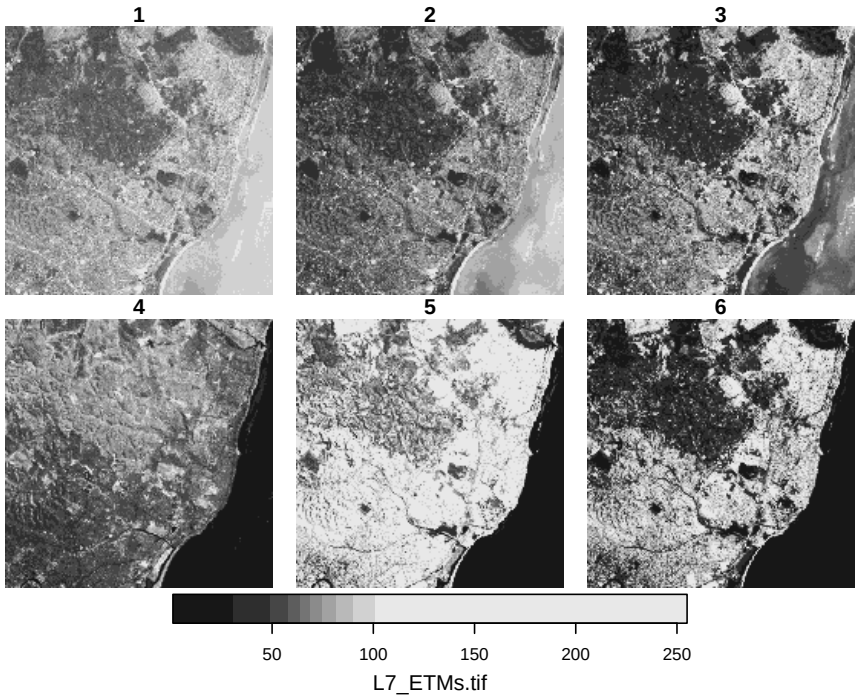


Figure 18: Les six bandes spectrales de l'image Landsat.

La dimension band valant 6, `plot()` affiche spontanément les six bandes en mosaïque, une vignette par bande.

4.2.2.2 Découper un cube : les crochets

On découpe un objet stars avec les crochets, en donnant des positions séparées par des virgules. La règle est simple : l'attribut vient en premier, puis chaque dimension dans l'ordre. Pour notre image, cela donne quatre positions : l'attribut, puis x, y et la bande.

```
bande4 <- l7[, , , 4]
zoom <- l7[, 1:100, 1:100, ]
```

La première ligne fixe la bande à 4 et laisse les autres positions vides : on garde toute l'image, mais sur la seule quatrième bande. La seconde fait l'inverse : elle restreint x et y aux cent premiers pixels et garde toutes les bandes, soit un carré découpé dans le coin de l'image. Une position laissée vide signifie donc « tout garder » pour cette dimension.

Deux compléments utiles. Par défaut, une dimension réduite à une seule valeur est conservée ; en ajoutant `drop = TRUE`, on la fait disparaître pour alléger l'objet. Et si l'on veut sélectionner non par position mais par valeur, par exemple toutes les colonnes situées entre deux coordonnées, la fonction `filter()` du tidyverse fonctionne directement sur un objet stars, ce qui illustre son intégration avec sf.

4.2.2.3 Calculer le long d'une dimension : `st_apply()`

`st_apply()` applique une fonction le long des dimensions choisies, sur le modèle de la fonction `apply()` de R (Pebesma et Bivand, 2023). Elle repose sur trois arguments :

- `X` est le cube de données à traiter ;
- `MARGIN` désigne les dimensions à conserver, par leur nom (`"x"`, `"y"`, `"band"`) ;
- `FUN` est la fonction à appliquer, par exemple `mean`.

Tout dépend des dimensions qu'on conserve. Si l'on garde x et y, le calcul se fait pour chaque pixel, à travers les bandes. La moyenne des six valeurs de chaque pixel s'écrit ainsi :

```
moyenne_pixel <- st_apply(l7, c("x", "y"), mean)
```

La dimension band disparaît, et il reste une image à une seule valeur par pixel. Si à l'inverse on conserve la bande, le calcul se fait pour chaque bande, à travers tous les pixels. On obtient alors une statistique par bande, et c'est la dimension spatiale qui disparaît :

```
moyenne_bande <- st_apply(l7, "band", mean)
```

Le résultat tient en six nombres, la valeur moyenne de chaque bande sur toute l'image. La règle est donc la suivante : les dimensions qu'on nomme sont celles qu'on garde, et le calcul s'effectue en parcourant celles qu'on laisse de côté. C'est l'outil de tous les indices spectraux, comme le NDVI de l'application 2, et de toutes les synthèses temporelles, comme une moyenne annuelle calculée sur une pile d'images mensuelles.

4.2.2.4 Passer d'un format à l'autre : `st_as_stars()` et `rast()`

```
alt_stars <- st_as_stars(alt)
alt_retour <- rast(alt_stars)
```

`st_as_stars()` convertit un objet terra en objet stars, et `rast()` fait le chemin inverse. Les deux packages cohabitent donc sans peine dans un même projet : terra pour les traitements lourds, stars quand la structure en cube ou le lien avec sf devient utile.

Le tableau suivant récapitule les arguments décisifs du package :

Fonction	Argument	Rôle
`read_stars()`	chemin du fichier	importe le cube de données
`[ ]`	attribut, x, y, bande	tranche le cube dimension par dimension
`st_apply()`	dimensions conservées	applique une fonction le long des autres dimensions
`st_as_stars()`	objet terra, sf ou matrice	convertit vers le format stars

4.2.3 Applications

Application 1. On veut prendre en main la structure d'un cube de données.

1. Importez l'image Landsat fournie avec stars ;
2. décrivez ses dimensions avec `st_dimensions()` ;
3. affichez uniquement sa première bande.

Correction.

L'image se charge avec `read_stars()`, et `st_dimensions()` en donne la structure.

```
l7 <- read_stars(system.file("tif/L7_ETMs.tif", package = "stars"))
st_dimensions(l7)
```

```
##      from  to  offset delta                     refsys point x/y
## x       1 349  288776  28.5 SIRGAS 2000 / UTM zone 25S FALSE [x]
## y       1 352 9120761 -28.5 SIRGAS 2000 / UTM zone 25S FALSE [y]
## band    1   6      NA    NA                           NA    NA
```

Le cube fait 349 sur 352 pixels, sur 6 bandes. On affiche ensuite la première bande seule.

```
plot(l7[, , , 1], main = "Bande 1")
```

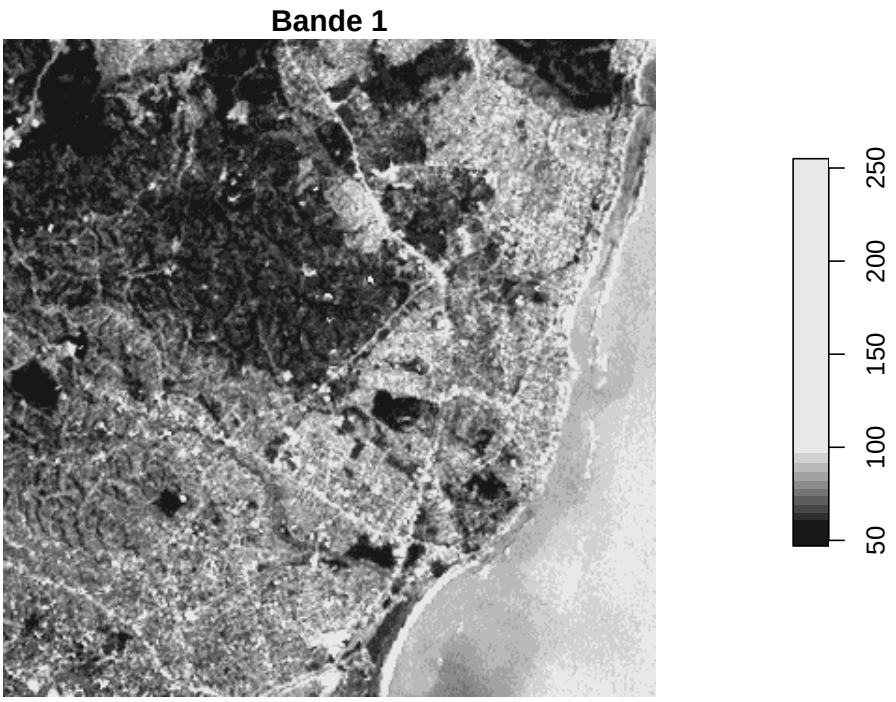


Figure 19: Première bande de l'image Landsat.

Pour n'afficher qu'une bande, on laisse les trois premières positions des crochets vides (tout x, tout y, tous les attributs) et on fixe la quatrième à 1.

Application 2. Le NDVI est un indice qui mesure la présence et la vigueur de la végétation à partir d'une image satellite ; on l'utilise pour suivre les cultures, les forêts ou la sécheresse. Il se calcule à partir de deux bandes, le rouge (bande 3) et le proche infrarouge (bande 4), comme leur différence divisée par leur somme.

1. Écrivez une fonction qui calcule cet indice à partir des valeurs d'un pixel ;
2. appliquez-la à tout le cube avec `st_apply()` ;
3. affichez le résultat sur une carte.

Correction.

La végétation réfléchit fortement le proche infrarouge et absorbe le rouge : des valeurs élevées de NDVI signalent donc une végétation dense.

```
ndvi_fun <- function(pixel) (pixel[4] - pixel[3]) / (pixel[4] + pixel[3])
ndvi <- st_apply(l7, c("x", "y"), ndvi_fun)
```

`st_apply()` reçoit pour chaque pixel le vecteur de ses six valeurs et notre fonction en tire l'indice. On affiche le résultat sur une carte, avec une palette de verts pour faire ressortir la végétation.

```
plot(ndvi, main = "NDVI", col = hcl.colors(11, "Greens", rev = TRUE))
```

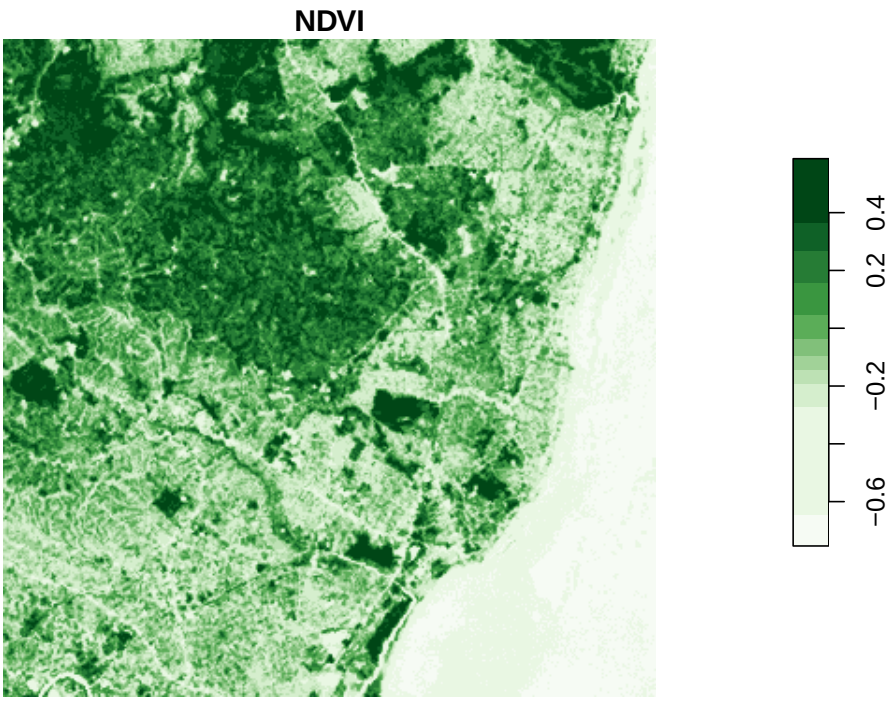


Figure 20: Indice de végétation NDVI ; le vert foncé marque la végétation dense.

L'indice varie de -1 à 1. La végétation ressort dans les valeurs hautes, en vert foncé, tandis que l'eau, ici la mer, prend des valeurs négatives et reste claire. En quelques lignes, on est passé d'une image satellite brute à un indicateur environnemental.

Chapitre 5 : Analyser les structures spatiales, spdep et gstat

Les chapitres précédents ont appris à importer, manipuler et cartographier les données spatiales. Reste la dernière étape de la chaîne, la plus statistique : mesurer et modéliser les structures qui se cachent dans l'espace. La carte du chapitre 3 suggérait déjà des regroupements, des comtés à forte mortalité voisins les uns des autres, des relevés de pollution concentrés près de la rivière. L'œil le devine, mais l'intuition ne suffit pas : il faut un test. Deux familles de méthodes répondent à deux questions distinctes, et chacune a son package.

La première question est celle des données surfaciques (chapitre 1) : les valeurs proches se ressemblent-elles plus que le hasard ne le voudrait ? C'est l'autocorrélation spatiale, domaine du package `spdep`. La seconde est celle des données continues : connaissant la valeur en quelques points, comment prédire celle des lieux non mesurés ? C'est l'interpolation, domaine de la géostatistique et du package `gstat`. Nous reprenons les deux jeux de données des chapitres précédents, les comtés de Caroline du Nord pour `spdep` et les relevés de la Meuse pour `gstat`, déjà chargés et reprojetés.

```
library(sf)
nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
nc_proj <- st_transform(nc, 32119)
nc_proj$taux <- 1000 * nc_proj$SID74 / nc_proj$BIR74
```

5.1 spdep : l'autocorrélation spatiale

5.1.1 Rôle

spdep (pour *spatial dependence*) est le package de référence pour l'analyse des données surfaciques, celles qui sont attachées à un découpage du territoire en zones contiguës. Publié par Roger Bivand, l'un des deux auteurs de `sp`, il met en œuvre les outils classiques de la statistique spatiale : les matrices de pondération qui définissent le voisinage, les tests d'autocorrélation globale comme l'indice de Moran, et leurs déclinaisons locales comme les indicateurs LISA.

L'idée que le package cherche à mesurer est la première loi de la géographie de Tobler, rencontrée au chapitre 1 : deux choses proches se ressemblent davantage que deux choses éloignées. Sur une carte de chômage communal, cela voudrait dire que les communes à fort chômage sont entourées d'autres communes à fort chômage, plutôt que dispersées au hasard. Cette ressemblance entre voisins porte un nom, l'autocorrélation spatiale, et tout l'enjeu de `spdep` est de la quantifier et de tester si elle est statistiquement significative.

Une précision de vocabulaire importante avant de commencer. L'autocorrélation peut être *positive*, quand les valeurs semblables se regroupent (le cas le plus fréquent), ou *négative*, quand les valeurs fortes sont systématiquement entourées de valeurs faibles, à la manière d'un damier. L'absence d'autocorrélation correspond à une répartition spatiale purement aléatoire. C'est cette dernière situation qui sert d'hypothèse de référence aux tests.

5.1.2 Les fonctions clés

5.1.2.1 Construire le voisinage : `poly2nb()` et `nb2listw()`

Avant toute mesure, il faut définir qui est voisin de qui. C'est l'étape fondatrice, et la plus lourde de conséquences : changer la définition du voisinage change tous les résultats qui suivent. La fonction `poly2nb()` construit une liste de voisinage à partir d'une couche de polygones, en considérant comme voisines deux zones qui partagent une frontière.

```
library(spdep)
voisins <- poly2nb(nc_proj, queen = TRUE)
voisins
```

```
## Neighbour list object:
## Number of regions: 100
```

```
## Number of nonzero links: 490
## Percentage nonzero weights: 4.9
## Average number of links: 4.9
```

L'objet décrit le réseau de voisinage : 100 comtés, et en moyenne 4,9 voisins par comté. L'argument `queen = TRUE` retient le voisinage « de la reine » (deux zones sont voisines si elles partagent ne serait-ce qu'un point, comme les déplacements de la reine aux échecs), par opposition à `queen = FALSE`, le voisinage « de la tour », plus strict, qui exige une frontière commune d'une certaine longueur.

La liste de voisinage indique qui touche qui, mais ne dit pas encore quel poids accorder à chaque voisin. C'est le rôle de `nb2listw()`, qui transforme la liste en une matrice de pondération exploitable par les tests :

```
poids <- nb2listw(voisins, style = "W", zero.policy = TRUE)
```

L'argument `style = "W"` produit une pondération standardisée par ligne : les poids de chaque comté somment à 1, de sorte qu'un comté entouré de deux voisins leur accorde chacun un poids de 0,5, tandis qu'un comté entouré de cinq voisins leur accorde 0,2 chacun. C'est le choix le plus courant, qui évite que les zones très entourées pèsent mécaniquement plus lourd dans l'analyse.

5.1.2.2 L'autocorrélation globale : `moran.test()`

L'indice de Moran résume en un seul nombre l'autocorrélation de toute la carte. Il varie schématiquement entre -1 et +1, sur le principe d'un coefficient de corrélation : proche de +1 quand les voisins se ressemblent fortement, proche de 0 en l'absence de structure, négatif en cas de répartition en damier. La fonction `moran.test()` calcule l'indice et le teste contre l'hypothèse nulle d'absence d'autocorrélation.

```
moran.test(nc_proj$taux, poids, zero.policy = TRUE)
```

```
##
##  Moran I test under randomisation
##
## data:  nc_proj$taux
## weights: poids
##
## Moran I statistic standard deviate = 3.7801, p-value = 7.839e-05
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.230910449      -0.010101010       0.004065134
```

La lecture se fait en deux temps. L'indice de Moran vaut ici environ 0,23, une valeur positive qui signale une tendance des comtés voisins à se ressembler. Reste à savoir si cette valeur est significative ou si elle pourrait résulter du hasard : c'est la p-value qui tranche. Inférieure à 0,05, elle conduit à rejeter l'hypothèse nulle d'absence d'autocorrélation. On conclut donc à une autocorrélation spatiale positive et significative : la mortalité infantile n'est pas distribuée au hasard sur le territoire, elle forme des regroupements géographiques.

L'hypothèse nulle mérite qu'on s'y arrête, car elle est au cœur de l'interprétation. Sous H0, les valeurs sont distribuées aléatoirement dans l'espace, sans lien avec la position. L'espérance de l'indice sous cette hypothèse n'est pas zéro mais une petite valeur négative, ici -1/(n-1), soit environ -0,01 pour 100 comtés. C'est à cette valeur de référence que l'indice observé est comparé.

5.1.2.3 Visualiser l'autocorrélation : `moran.plot()`

Le diagramme de Moran offre une lecture graphique de cette même information. Il porte en abscisse la valeur de chaque comté et en ordonnée la moyenne de ses voisins (le décalage spatial). La pente de la droite de régression vaut exactement l'indice de Moran.

```
moran.plot(nc_proj$taux, poids, zero.policy = TRUE,
           xlab = "Taux de mortalité",
           ylab = "Moyenne des voisins")
```

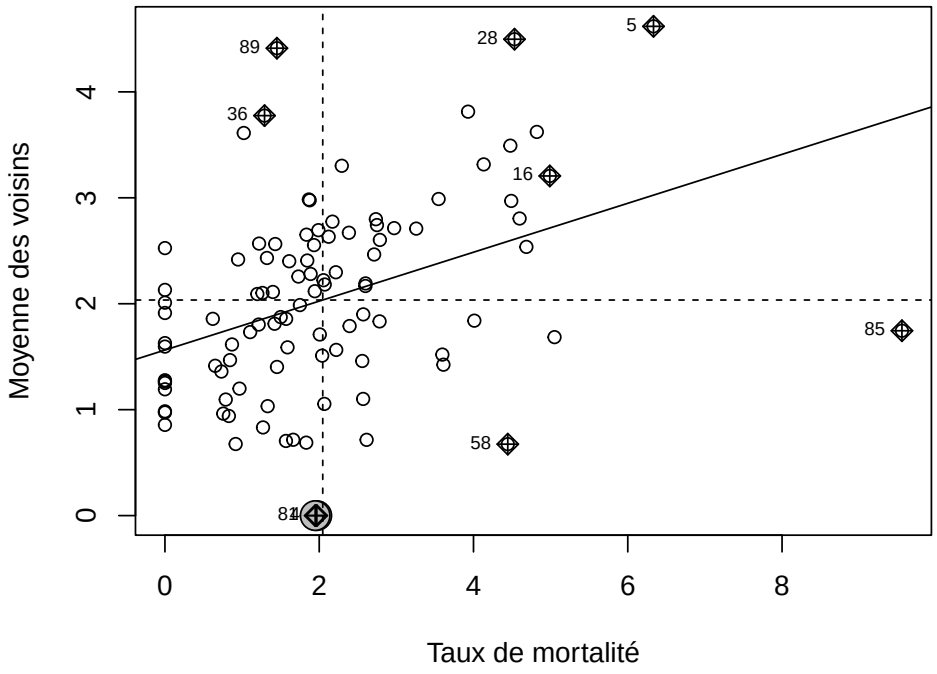


Figure 21: Diagramme de Moran du taux de mortalité infantile.

Le nuage se répartit en quatre quadrants délimités par les moyennes. Le quadrant en haut à droite rassemble les comtés à forte valeur entourés de voisins à forte valeur (regroupements hauts), celui en bas à gauche les valeurs faibles entourées de faibles (regroupements bas). Une pente positive, modérée mais nette, confirme visuellement l'autocorrélation détectée par le test.

5.1.2.4 L'autocorrélation locale : `localmoran()` et les LISA

L'indice de Moran global a une limite : il résume toute la carte en un nombre, et masque les disparités locales. Une carte peut très bien combiner une grande zone de regroupements hauts et une autre de regroupements bas, sans que le global ne le révèle. Les indicateurs LISA (*Local Indicators of Spatial Association*), proposés par Luc Anselin, décomposent l'indice global en une contribution par zone, ce qui permet de localiser les regroupements sur la carte.

```
lmoran <- localmoran(nc_proj$taux, poids, zero.policy = TRUE)
head(lmoran, 3)
```

```
##          Ii          E.Ii     Var.Ii      Z.Ii Pr(z != E(Ii))
## 1 0.6310748 -0.0052538389 0.17065261 1.5403698     0.12347025
## 2 0.6623095 -0.0172474220 0.55346771 0.9134394     0.36101152
## 3 0.2611271 -0.0009386677 0.01799019 1.9538552     0.05071835
```

Chaque comté reçoit son propre indice `Ii`, sa significativité (`Pr`) et un score `Z`. À partir de là, on classe chaque zone significative dans l'un des quatre types : High-High (forte valeur parmi des fortes, un « point chaud »), Low-Low (faible parmi des faibles, un « point froid »), et les deux cas atypiques High-Low et Low-High (une valeur qui détonne dans son voisinage). Ce sont ces catégories que l'on cartographie pour donner à voir les regroupements, exactement la carte que nous produirons en partie II sur la Normandie.

5.1.3 Applications

Application 1. Construisez la liste de voisinage des comtés de Caroline du Nord selon le critère de la reine, puis la matrice de pondération standardisée par ligne. Combien de voisins compte en moyenne chaque comté ?

Correction.

```
library(spdep)
voisins <- poly2nb(nc_proj, queen = TRUE)
poids <- nb2listw(voisins, style = "W", zero.policy = TRUE)
voisins
```

```
## Neighbour list object:
## Number of regions: 100
## Number of nonzero links: 490
## Percentage nonzero weights: 4.9
## Average number of links: 4.9
```

La construction du voisinage est l'étape qui conditionne tout le reste. Chaque comté a en moyenne 4,9 voisins, ce qui est cohérent avec un découpage administratif où chaque unité est entourée d'une poignée d'autres. La standardisation par ligne (`style = "W"`) garantit ensuite que tous les comtés pèsent le même poids total dans l'analyse, quel que soit leur nombre de voisins.

Application 2. Testez l'autocorrélation spatiale globale du taux de mortalité infantile à l'aide de l'indice de Moran. Que concluez-vous ?

Correction.

```
moran.test(nc_proj$taux, poids, zero.policy = TRUE)
```

```
##
##  Moran I test under randomisation
##
## data:  nc_proj$taux
## weights: poids
##
## Moran I statistic standard deviate = 3.7801, p-value = 7.839e-05
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.230910449      -0.010101010       0.004065134
```

L'indice vaut environ 0,23, positif, et la p-value est très inférieure à 0,05. On rejette donc l'hypothèse nulle d'une répartition aléatoire : la mortalité infantile présente une autocorrélation spatiale positive et significative. Autrement dit, un comté à forte mortalité a de bonnes chances d'être entouré de comtés également touchés. La structure que la carte du chapitre 3 laissait deviner est maintenant établie statistiquement.

Application 3. Calculez les indicateurs locaux de Moran (LISA) et déterminez combien de comtés présentent une autocorrélation locale significative au seuil de 5 %.

Correction.

```
lmoran <- localmoran(nc_proj$taux, poids, zero.policy = TRUE)
significatifs <- sum(lmoran[, "Pr(z != E(Ii))"] < 0.05)
significatifs
```

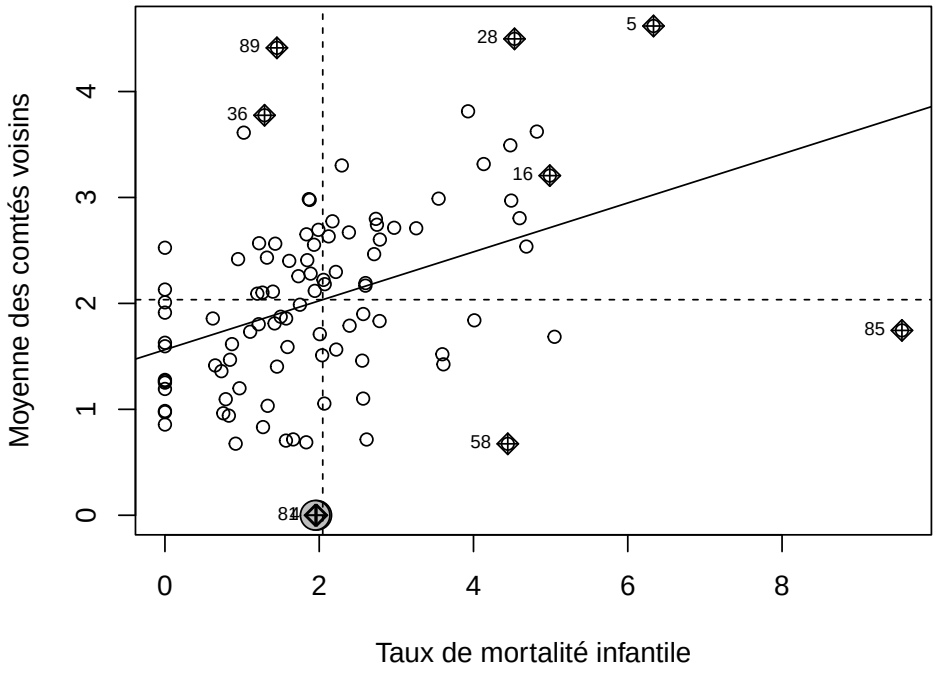
```
## [1] 10
```

Les LISA décomposent l'indice global comté par comté. Seule une partie des comtés ressort comme significative : ce sont eux qui forment les véritables regroupements, le reste de la carte ne se distinguant pas du hasard. Cartographier uniquement ces comtés significatifs, en les coloriant selon leur type (High-High, Low-Low…), donne la carte des regroupements qui constitue l'aboutissement de l'analyse, et que nous reproduirons sur les données normandes.

Application 4. Tracez le diagramme de Moran du taux de mortalité et reliez sa pente à l'indice global calculé plus haut.

Correction.

```
moran.plot(nc_proj$taux, poids, zero.policy = TRUE,
           xlab = "Taux de mortalité infantile",
           ylab = "Moyenne des comtés voisins")
```



Le diagramme confirme visuellement le test. La pente de la droite de régression est exactement l'indice de Moran, environ 0,23, d'où une droite à pente positive modérée. Les comtés situés dans les quadrants haut-droit et bas-gauche, alignés avec la droite, sont ceux qui nourrissent l'autocorrélation positive ; les points isolés dans les autres quadrants sont les exceptions locales que les LISA repéreront.

5.2 gstat : interpolation et géostatistique

5.2.1 Rôle

gstat aborde l'autre grande famille de données du chapitre 1, les données continues, celles qui possèdent une valeur en tout point du territoire mais que l'on ne mesure qu'à quelques endroits. La température, la concentration d'un polluant ou l'altitude en sont des exemples : il existe une valeur partout, mais les relevés sont rares et dispersés. La question devient alors prédictive : comment estimer la valeur là où aucune mesure n'a été faite ?

Le package, écrit par Edzer Pebesma (déjà auteur de `sp` et `sf`), met en œuvre les outils de la géostatistique. Le principe directeur reste celui de Tobler : pour estimer une valeur en un point, on s'appuie sur les mesures voisines, en accordant d'autant plus de poids qu'elles sont proches. Reste à formaliser ce « d'autant plus que c'est proche », et c'est le rôle du variogramme, l'outil central du package.

Nous travaillons sur le jeu `meuse`, les 155 relevés de concentration en métaux lourds de la plaine de la Meuse, déjà rencontrés au chapitre 2. La variable d'intérêt est la concentration en zinc.

```
library(sf)
data(meuse, package = "sp")
meuse_sf <- st_as_sf(meuse, coords = c("x", "y"), crs = 28992)
```

5.2.2 Les fonctions clés

5.2.2.1 Décrire la structure spatiale : `variogram()`

Le variogramme est au cœur de la géostatistique. Il décrit comment la ressemblance entre deux mesures se dégrade à mesure qu'elles s'éloignent. Concrètement, pour chaque paire de points, on calcule l'écart de leurs valeurs et la distance qui les sépare, puis on moyenne ces écarts par tranche de distance. La fonction `variogram()` produit ce nuage.

```
library(gstat)
vario <- variogram(log(zinc) ~ 1, data = meuse_sf)
plot(vario)
```

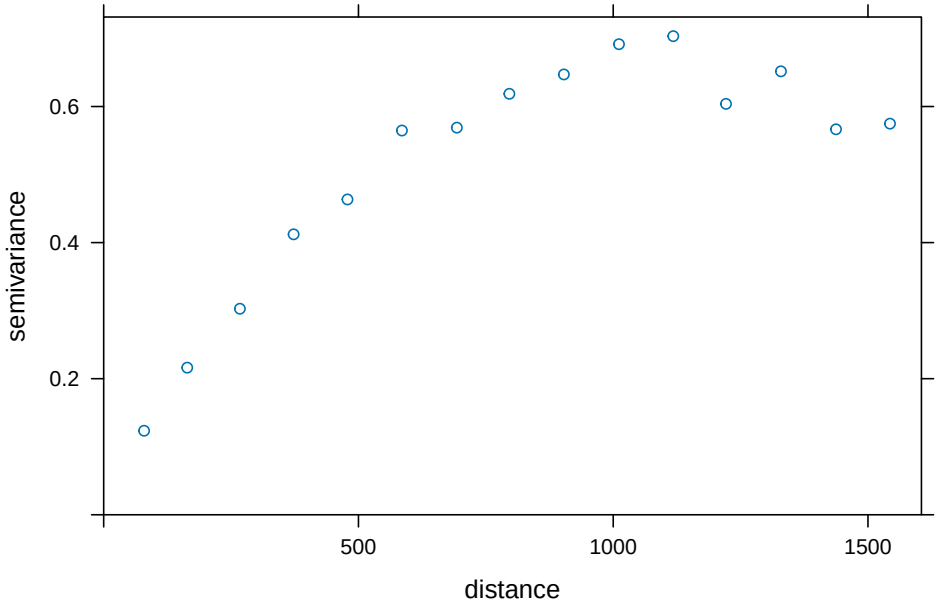


Figure 22: Semi-variogramme expérimental du zinc (échelle log).

On modélise ici le logarithme du zinc, car la variable est très asymétrique, ce qui est courant pour des concentrations. La formule `log(zinc) ~ 1` indique que l'on étudie la seule structure spatiale de la variable, sans la relier à d'autres facteurs. Le variogramme empirique croît typiquement avec la distance : plus deux points sont éloignés, plus leurs concentrations diffèrent, jusqu'à un palier au-delà duquel la distance n'apporte plus d'information.

Trois grandeurs résument un variogramme, et leur vocabulaire revient sans cesse en géostatistique. La *pépite* (nugget) est la valeur à l'origine, l'écart résiduel entre points théoriquement confondus, qui traduit le bruit de mesure et la variabilité à très courte distance. Le *palier* (sill) est la valeur plafond atteinte aux grandes distances. La *portée* (range) est la distance à partir de laquelle le palier est atteint, c'est-à-dire la distance au-delà de laquelle deux mesures n'ont plus rien à voir l'une avec l'autre.

5.2.2.2 Ajuster un modèle : `fit.variogram()`

Le variogramme empirique est un nuage de points ; pour s'en servir dans une prédiction, il faut l'ajuster par une courbe théorique. La fonction `fit.variogram()` cale un modèle (sphérique, exponentiel, gaussien…) sur le nuage, en estimant ses trois paramètres.

```
modele <- fit.variogram(vario, model = vgm(1, "Sph", 900, 1))
modele
```

```
##   model      psill    range
## 1   Nug 0.05066243   0.0000
## 2   Sph 0.59060780 897.0209
```

L'appel `vgm(1, "Sph", 900, 1)` fournit les valeurs de départ de l'ajustement : un modèle sphérique (`"Sph"`), un palier initial de 1, une portée initiale de 900 mètres et une pépite de 1. La fonction affine ces valeurs pour coller au mieux au variogramme empirique. La portée ajustée, ici de l'ordre de 900 mètres, indique la distance d'influence : deux relevés distants de plus de 900 mètres peuvent être considérés comme indépendants.

5.2.2.3 Prédire par krigeage : `krige()`

Le krigeage est la méthode d'interpolation phare de la géostatistique. À la différence d'une moyenne pondérée naïve, il exploite le variogramme pour pondérer les mesures voisines de façon optimale, et il fournit en prime une estimation de l'incertitude de chaque prédiction. La fonction `krige()` réalise l'interpolation sur une grille de points couvrant la zone d'étude.

```
data(meuse.grid, package = "sp")
grille_sf <- st_as_sf(meuse.grid, coords = c("x", "y"), crs = 28992)
prediction <- krige(log(zinc) ~ 1, meuse_sf, grille_sf, model = modele)
```

L'objet `meuse.grid`, livré avec `sp`, fournit une grille régulière couvrant la plaine. La fonction `krige()` y prédit la concentration en zinc en chaque cellule, à partir des 155 relevés et du modèle de variogramme. Le résultat contient deux colonnes : la prédiction elle-même (`var1.pred`) et sa variance (`var1.var`), qui mesure la confiance accordée à chaque estimation, plus faible près des points mesurés et plus forte dans les zones vides.

5.2.2.4 Cartographier la prédiction

La prédiction krigée se cartographie comme n'importe quelle couche, par exemple avec `ggplot2` :

```
library(ggplot2)
ggplot(prediction) +
  geom_sf(aes(color = var1.pred)) +
  scale_color_viridis_c() +
  labs(title = "Zinc interpolé par krigeage (log)",
       color = "log(zinc)") +
  theme_void()
```

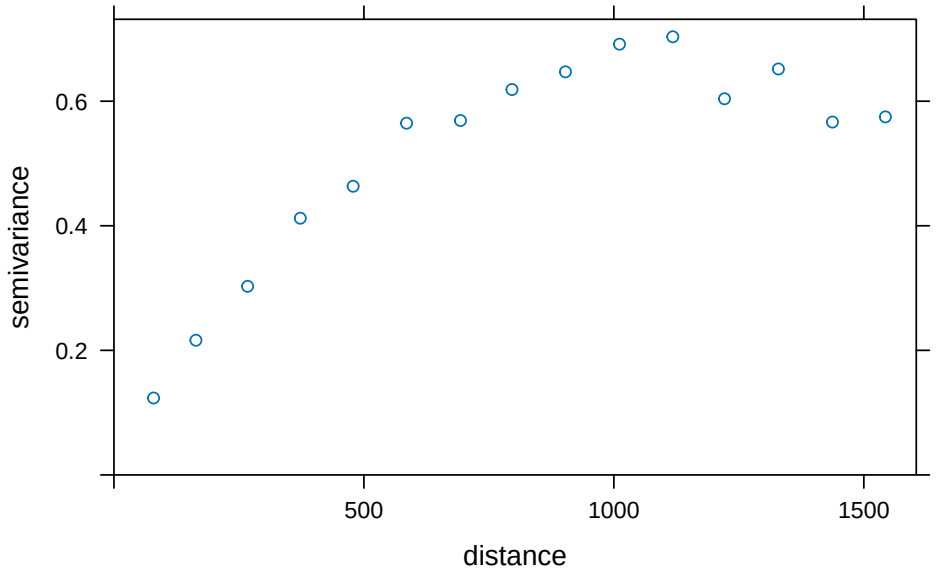
La carte obtenue couvre toute la plaine d'un dégradé continu, alors que les données de départ n'étaient que 155 points isolés. C'est tout l'apport du krigeage : passer d'une connaissance ponctuelle à une surface continue, en s'appuyant sur la structure spatiale décrite par le variogramme.

5.2.3 Applications

Application 1. Calculez et tracez le variogramme empirique du logarithme de la concentration en zinc. Pourquoi modéliser le logarithme plutôt que la variable brute ?

Correction.

```
library(gstat)
vario <- variogram(log(zinc) ~ 1, data = meuse_sf)
plot(vario)
```



Le variogramme croît avec la distance puis se stabilise, signe d'une structure spatiale nette : les relevés proches se ressemblent, les relevés éloignés non. On modélise le logarithme parce que la concentration en zinc est fortement asymétrique, avec quelques valeurs très élevées près de la rivière. Le passage au log symétrise la distribution et stabilise la variance, condition d'un krigeage de bonne qualité.

Application 2. Ajustez un modèle sphérique sur ce variogramme et interprétez la portée obtenue.

Correction.

```
modele <- fit.variogram(vario, model = vgm(1, "Sph", 900, 1))
modele
```

```
##   model      psill    range
## 1   Nug 0.05066243   0.0000
## 2   Sph 0.59060780 897.0209
```

La portée ajustée, de l'ordre de 900 mètres, est l'information la plus parlante : au-delà de cette distance, deux relevés n'apportent plus d'information l'un sur l'autre. C'est cette distance qui déterminera quels points voisins contribuent à la prédiction en un lieu donné. La pépite, faible, indique que la variabilité à très courte distance reste limitée.

Application 3. Réalisez le krigeage de la concentration en zinc sur la grille `meuse.grid` et identifiez les colonnes du résultat.

Correction.

```
data(meuse.grid, package = "sp")
grille_sf <- st_as_sf(meuse.grid, coords = c("x", "y"), crs = 28992)
prediction <- krige(log(zinc) ~ 1, meuse_sf, grille_sf, model = modele)
names(prediction)
```

Le krigeage produit une prédiction en chaque cellule de la grille. La colonne `var1.pred` donne la valeur estimée du log de zinc, la colonne `var1.var` la variance de cette estimation. Cette seconde colonne est précieuse : elle dit où la prédiction est fiable (près des relevés) et où elle l'est moins (dans les zones non échantillonnées), information qu'aucune interpolation naïve ne fournit.

Application 4. Cartographiez la prédiction krigée et commentez l'apport par rapport aux 155 points de départ.

Correction.

```
library(ggplot2)
ggplot(prediction) +
  geom_sf(aes(color = var1.pred)) +
  scale_color_viridis_c(option = "plasma") +
  labs(title = "Concentration en zinc interpolée (log)",
       subtitle = "Krigeage ordinaire, plaine de la Meuse",
       color = "log(zinc)",
       caption = "Source : jeu meuse, package sp") +
  theme_void()
```

Là où l'on ne disposait que de 155 relevés dispersés, le krigeage produit une surface continue couvrant toute la plaine. Les fortes concentrations se concentrent le long de la rivière, confirmant et précisant l'intuition du chapitre 2. C'est le passage du ponctuel au continu, but même de la géostatistique : non pas inventer de l'information, mais l'étendre de façon rigoureuse en s'appuyant sur la structure spatiale mesurée par le variogramme.

Chapitre 6 : Cartographier et analyser l'offre de formation en Normandie

Ce chapitre applique la chaîne de traitement de la partie I à un cas réel : la répartition de l'offre de formation supérieure en Normandie, à partir des données Parcoursup 2024. Trois packages sont mobilisés, chacun pour son maillon : `sf` prépare les données, `tmap` les cartographie, `spdep` analyse leur structure spatiale.

Pourquoi cette étude

Chaque année, des centaines de milliers de lycéens formulent leurs vœux d'orientation sur Parcoursup. Mais tous ne partent pas avec les mêmes chances. Pour un jeune vivant dans une commune éloignée de toute université, poursuivre des études suppose souvent de quitter son domicile, avec un coût financier et personnel que toutes les familles ne peuvent assumer de la même façon. La répartition des formations sur le territoire devient alors un facteur d'inégalité des chances : là où l'offre est proche, l'accès est facilité ; là où elle est absente, il se paie en mobilité. Comprendre comment cette offre se répartit, c'est donc éclairer une des dimensions les moins visibles des inégalités scolaires.

L'accès à l'enseignement supérieur ne dépend pas seulement des résultats scolaires : il dépend aussi de la géographie. Un lycéen n'a pas les mêmes possibilités selon qu'il habite une métropole dotée de nombreuses formations ou une commune éloignée du premier campus. Parcoursup, qui centralise toutes les formations du supérieur, offre une trace précise de cette géographie.

Ce chapitre pose une question simple : **l'offre de formation supérieure est-elle également répartie sur le territoire normand, ou se concentre-t-elle sur quelques pôles ?** Y répondre, c'est mesurer une première forme d'inégalité, celle de l'offre, avant même de parler des candidats.

Ce chapitre poursuit trois objectifs. Il s'agit d'abord de cartographier l'offre à l'échelle communale. Vient ensuite une question plus statistique : cette répartition relève-t-elle d'une vraie structure spatiale, ou pourrait-elle tenir du hasard ? L'indice de Moran permet d'y répondre. Les indicateurs locaux LISA aident enfin à localiser précisément les regroupements. Ces trois étapes s'appuient sur les hypothèses suivantes :

- **H1.** L'offre se concentre sur les grandes agglomérations, en premier lieu Caen, Rouen et Le Havre ;
- **H2.** Cette concentration n'est pas aléatoire mais spatialement structurée : les communes dotées sont proches les unes des autres et forment des pôles ;
- **H3.** La majorité du territoire, notamment rural, ne propose aucune offre communale, l'offre étant l'apanage d'un petit nombre de communes.

Ce que disent les travaux existants

La dimension territoriale des inégalités d'accès au supérieur est un objet de recherche ancien. Dès 1990, Thumerelle et Dumont montraient, sur l'académie de Lille, que la répartition spatiale des établissements engendre des inégalités d'accès longtemps moins étudiées que les inégalités sociales, et que les formations courtes comme les sections de techniciens supérieurs sont bien plus dispersées sur le territoire que les universités, fortement concentrées.

Ce constat reste d'actualité, et à l'échelle nationale. Dans un Focus du Conseil d'analyse économique de 2025, Anaïs Fabre établit qu'une dizaine de départements métropolitains ne proposent aucune licence ni classe préparatoire, et que la moitié des capacités d'accueil dans ces formations se concentre dans une douzaine de départements seulement. Elle montre aussi que la moitié des élèves n'ont aucune licence dans un rayon de dix kilomètres autour de leur domicile, tandis que les BTS et les DUT sont bien plus également répartis. La distance à la formation pèse alors réellement sur les choix d'orientation.

L'arrivée de Parcoursup en 2018 a relancé ces questions. Les travaux de Frouillou, notamment avec Couto et Bugeja-Bloch, ont analysé comment la plateforme redistribue les vœux dans un espace de formation hiérarchisé, en insistant sur la dimension à la fois sociale et spatiale des inégalités d'accès.

Deux enseignements se dégagent pour notre étude. D'abord, la concentration de l'offre sur quelques pôles est un fait attendu, documenté du niveau national jusqu'à l'échelle régionale : notre analyse vient l'illustrer sur la Normandie. Ensuite, ces travaux distinguent nettement l'offre de formation, c'est-à-dire sa localisation, et l'accessibilité réelle, qui dépend de la distance et de la mobilité. Notre travail porte sur l'offre ; l'accessibilité en constitue le prolongement naturel, qu'on esquissera en conclusion.

Les données et le terrain

Deux sources sont croisées. D'une part, le jeu Parcoursup 2024 du ministère de l'Enseignement supérieur, qui recense chaque formation proposée sur la plateforme, géolocalisée par un point et décrite par plus d'une centaine de variables (capacité d'accueil, vœux reçus, admis, part de boursiers, taux d'accès). D'autre part, le fond communal de l'IGN (contours des communes issus d'Admin Express, diffusés au format shapefile), qui fournit le découpage surfacique nécessaire à l'agrégation et à l'analyse de voisinage.

Le terrain retenu est celui des cinq départements normands : le Calvados (14), l'Eure (27), la Manche (50), l'Orne (61) et la Seine-Maritime (76). Après filtrage, l'étude porte sur **615 formations géolocalisées** (618 formations normandes, dont trois sans coordonnées), sur un territoire comptant **2 651 communes**.

Préparer les données avec sf

La préparation suit exactement la logique du chapitre 1 : importer, reprojeter, joindre. C'est l'étape la plus longue en pratique, et celle qui conditionne la justesse de tout le reste.

Importer et filtrer

```
library(sf)
library(dplyr)

deps_normands <- c("14", "27", "50", "61", "76")

# Import du jeu national Parcoursup (points en WGS 84), puis filtrage Normandie
formations <- st_read("fr-esr-parcoursup_2024.geojson", quiet = TRUE)

form_norm <- formations %>%
  filter(region_etab_aff == "Normandie") %>%   # 618 formations normandes
  filter(!st_is_empty(.)) %>%                   # retirer les géométries vides
  mutate(taux_acces = as.numeric(taux_acces_ens))  # taux stocké en texte -> numérique

nrow(form_norm)
```

```
## [1] 615
```

Le jeu national compte plus de 14 000 formations ; on le filtre sur la variable `region_etab_aff` pour ne garder que la Normandie, soit **618 formations**. Deux précautions accompagnent ce filtrage : on écarte les formations sans coordonnées avec `st_is_empty()`, ce qui en retire trois et laisse **615 formations géolocalisées** (c'est le chiffre affiché par `nrow()`), et on convertit le taux d'accès, stocké en texte dans le fichier, avec `as.numeric()`.

Charger et assembler le fond communal

```
# Fond communal national IGN (shapefile), filtré sur les 5 départements normands
communes <- st_read("shapefiles/COMMUNE.shp", quiet = TRUE) %>%
  filter(INSEE_DEP %in% deps_normands) %>%
  select(INSEE_COM, NOM, INSEE_DEP, POPULATION)

nrow(communes)
```

```
## [1] 2651
```

Le shapefile national contient les 34 935 communes françaises. On le filtre sur les cinq départements normands via la colonne `INSEE_DEP`, ce qui donne **2 651 communes**. Chaque commune porte son code INSEE (`INSEE_COM`), son nom (`NOM`) et sa population (`POPULATION`), cette dernière ouvrant la voie à une lecture rapportée aux habitants.

Reprojeter en Lambert 93

```
# Réflexe du chapitre 1 : passer en Lambert 93 avant tout calcul
form_l93     <- st_transform(form_norm, 2154)
communes_l93 <- st_transform(communes, 2154)
```

Les deux couches arrivent en WGS 84 (degrés). Avant toute jointure spatiale ou mesure de voisinage, on les reprojette en Lambert 93 (EPSG 2154), le système projeté légal en France métropolitaine, exprimé en mètres. C'est la condition d'un calcul spatial correct, comme le rappelait le chapitre 1.

Joindre les formations aux communes

C'est le cœur de la préparation. Chaque formation est un point ; on veut savoir dans quelle commune il tombe. C'est une jointure spatiale, réalisée avec `st_join()` et le prédicat `st_within` (« quel polygone contient ce point ? »).

```
form_comm <- st_join(form_l93, communes_l93[, c("INSEE_COM", "NOM")],
                     join = st_within)

# Contrôle : combien de formations n'ont pas trouvé leur commune ?
sum(is.na(form_comm$INSEE_COM))
```

```
## [1] 0
```

Ce contrôle indique combien de formations n'ont pas trouvé de commune. Il doit renvoyer un nombre très faible, idéalement zéro : cela signifie que les points tombent bien à l'intérieur des polygones, et donc que les deux couches sont correctement alignées dans le même système de coordonnées. Si quelques points restent non rattachés (une géométrie communale imprécise, un point sur une frontière), on peut les rattacher à la commune la plus proche avec `st_nearest_feature()`.

Agréger l'offre par commune

Une formation est une ligne ; on veut désormais un indicateur par commune. On compte le nombre de formations de chaque commune, puis on rattache ce comptage au fond communal complet, en attribuant zéro aux communes sans offre.

```
# Comptage des formations par code commune
offre <- form_comm %>%
  st_drop_geometry() %>%
  count(INSEE_COM, name = "nb_formations")

# Jointure attributaire sur le fond communal ; les communes absentes = 0
communes_l93 <- communes_l93 %>%
  left_join(offre, by = "INSEE_COM") %>%
  mutate(nb_formations = tidyr::replace_na(nb_formations, 0))

# Quelques chiffres de cadrage
sum(communes_l93$nb_formations)              # total formations
```

```
## [1] 615
```

```
sum(communes_193$nb_formations > 0)           # communes avec offre
```

```
## [1] 76
```

```
mean(communes_193$nb_formations == 0) * 100   # % de communes sans offre
```

```
## [1] 97.13316
```

Les trois lignes affichent, dans l'ordre, le total des formations rattachées, le nombre de communes qui en proposent au moins une, et le pourcentage de communes sans aucune offre. Le chiffre surprend même en le connaissant à l'avance : sur **615 formations**, à peine **76 communes** en accueillent au moins une, soit 2,9 % du territoire communal normand. Autrement dit, **97,1 % des communes ne proposent aucune formation post-bac**. Ce constat, à lui seul, va déjà dans le sens de l'hypothèse H3.

Cartographier l'offre avec tmap

Les données sont prêtes. On les donne à voir en respectant les règles de sémiologie du chapitre 1 : le nombre de formations est un **effectif brut**, il appelle donc soit une choroplèthe en classes discrètes, soit des cercles proportionnels. On commence par la choroplèthe, complétée par le contour départemental et les noms des principaux repères géographiques.

```
# Sélection des communes essentielles à étiqueter sur les cartes
villes_a_labeliser <- communes_193 %>%
  filter(NOM %in% c("Caen", "Hérouville-Saint-Clair", "Ifs", "Mondeville",
                    "Rouen", "Mont-Saint-Aignan", "Saint-Étienne-du-Rouvray",
                    "Sotteville-lès-Rouen", "Le Havre",
                    "Alençon", "Damigny",
                    "Cherbourg-en-Cotentin", "Évreux", "Saint-Lô"))
```

```
library(tmap)

# Contour des départements pour l'habillage (shapefile IGN filtré Normandie)
departements <- st_read("shapefiles/DEPARTEMENT.shp", quiet = TRUE) %>%
  filter(INSEE_DEP %in% deps_normands) %>%
  st_transform(2154)

tm_shape(communes_193) +
  tm_polygons(fill = "nb_formations",
              fill.scale = tm_scale_intervals(style = "jenks", n = 5,
                                              values = "brewer.reds"),
              fill.legend = tm_legend(title = "Nb de formations"),
              col = "white", lwd = 0.1) +

  # Noms des départements
  tm_shape(departements) +
  tm_borders(col = "black", lwd = 0.8) +
  tm_text(text = "NOM", size = 0.8, col = "black", fontface = "bold",
          shadow = TRUE, bg.color = "white", bg.alpha = 0.5) +

  # Noms des communes essentielles
  tm_shape(villes_a_labeliser) +
  tm_text(text = "NOM", size = 0.6, col = "black", fontface = "bold",
```

```
          shadow = TRUE, bg.color = "white", bg.alpha = 0.6) +

  tm_title("Offre de formation supérieure en Normandie (Parcoursup 2024)") +
  tm_scalebar(position = c("left", "bottom")) +
  tm_credits("Sources : Parcoursup 2024, IGN Admin Express")
```

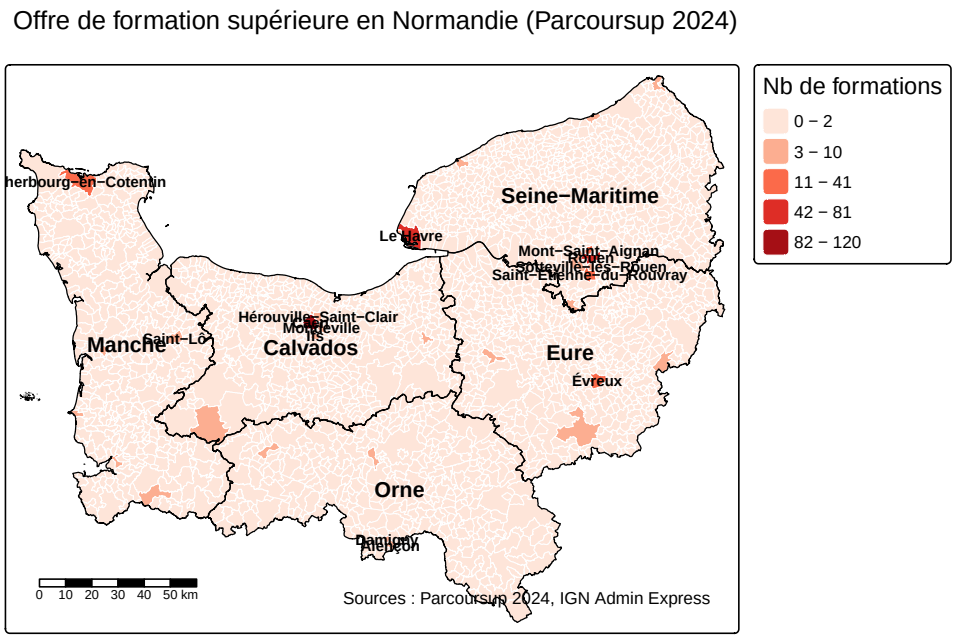


Figure 23: Offre de formation Parcoursup par commune, Normandie 2024

La carte parle d'elle-même. Trois taches foncées ressortent : l'agglomération de **Caen** (14), le couple **Rouen–Mont-Saint-Aignan** (76) et **Le Havre** (76). Le reste du territoire, en teinte pâle, correspond presque exclusivement à des communes sans aucune offre (la première classe, 0 à 2 formations, est dominée par les communes à zéro, qui représentent 97,1 % du total). À elles seules, les trois premières communes concentrent **42.6 % des formations** de la région.

Pour respecter pleinement la sémiologie, on peut doubler la lecture avec des cercles proportionnels au nombre de formations, qui rendent le volume sans la distorsion de surface propre aux grandes communes rurales :

```
tm_shape(communes_193) +
  tm_polygons(fill = "grey95", col = "white", lwd = 0.1) +

  tm_shape(departements) +
  tm_borders(col = "grey40", lwd = 0.6) +
  tm_text(text = "NOM", size = 0.8, col = "grey30", fontface = "bold") +

  tm_shape(communes_193[communes_193$nb_formations > 0, ]) +
  tm_symbols(size = "nb_formations", fill = "steelblue", fill_alpha = 0.6,
             size.legend = tm_legend(title = "Nb de formations")) +

  tm_shape(villes_a_labeliser) +
  tm_text(text = "NOM", size = 0.6, col = "black", fontface = "bold",
```

```
          shadow = TRUE, bg.color = "white", bg.alpha = 0.5) +

  tm_title("Localisation et volume de l'offre (Normandie, 2024)")
```

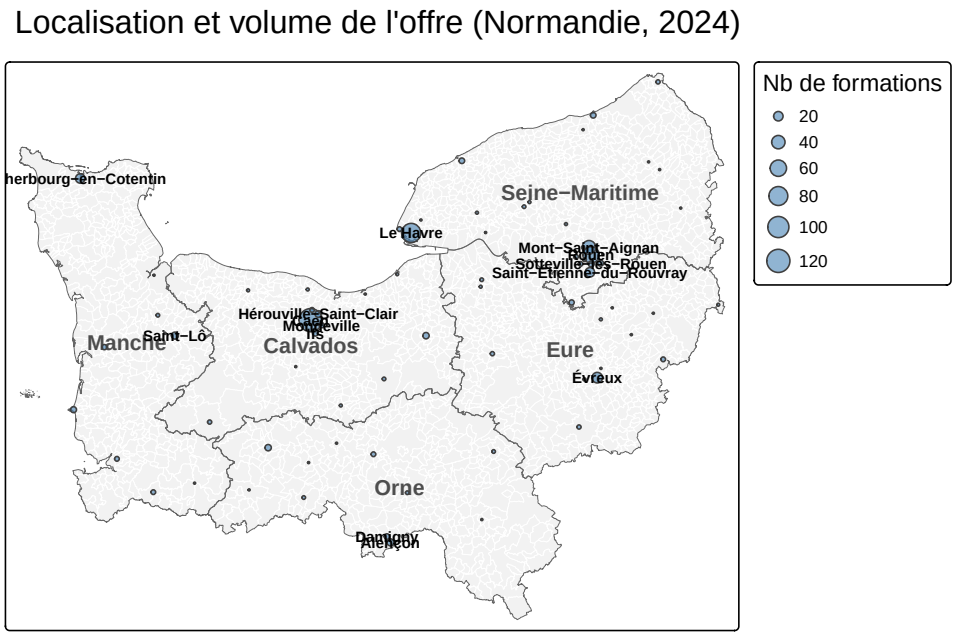


Figure 24: Nombre de formations par commune (cercles proportionnels)

Analyser la structure spatiale avec spdep

La carte suggère une concentration. Il reste à la démontrer statistiquement : cette concentration est-elle réelle, ou pourrait-elle résulter du hasard ? C'est la question de l'autocorrélation spatiale, et c'est le rôle de `spdep`.

Construire la matrice de voisinage

Toute analyse spatiale commence par définir « qui est voisin de qui ». On construit une matrice de contiguïté de type *Queen* (deux communes sont voisines si elles partagent au moins un point de frontière), puis on la normalise en ligne pour que les poids de chaque commune somment à 1.

```
library(spdep)

# Voisinage par contiguité Queen
voisins <- poly2nb(communes_193, queen = TRUE)

# Passage en liste de poids, normalisée en ligne (style = "W")
poids <- nb2listw(voisins, style = "W", zero.policy = TRUE)
```

L'argument `zero.policy = TRUE` autorise l'existence de communes sans voisin (îles, enclaves), qui recevraient sinon une erreur.

L'indice de Moran global

L'indice de Moran mesure, en un seul chiffre compris entre -1 et +1, si les valeurs voisines se ressemblent. Positif, il signale un regroupement des valeurs semblables ; nul, une répartition aléatoire. La significativité

est évaluée par un test de permutations : on rebat au hasard les valeurs sur les communes des centaines de fois, et on regarde si l'indice observé sort du lot.

```
set.seed(42)
moran_test <- moran.mc(communes_l93$nb_formations, poids,
                       nsim = 999, zero.policy = TRUE)
moran_test
```

```
##
##  Monte-Carlo simulation of Moran I
##
## data:  communes_l93$nb_formations
## weights: poids
## number of simulations + 1: 1000
##
## statistic = 0.085291, observed rank = 995, p-value = 0.005
## alternative hypothesis: greater
```

Le résultat donne un **indice de Moran de 0,085**, avec une **p-value de 0,005** (rang observé 995 sur 1 000 permutations). L'interprétation demande de la nuance, et c'est là tout l'intérêt du cas normand. L'indice est **positif et hautement significatif** : la concentration de l'offre n'est pas le fruit du hasard, les communes bien dotées sont bien entourées d'autres communes dotées. Mais sa valeur reste **modérée**, et cette modération est elle-même parlante : comme 97 % des communes sont à zéro, l'immense majorité des couples de voisins sont des couples « zéro-zéro », qui diluent l'intensité de l'indice. Autrement dit, le Moran modéré ne traduit pas une absence de structure, mais l'ampleur du désert universitaire qui entoure quelques pôles denses.

On notera au passage le piège classique de la p-value, déjà rencontré en statistique : ici, une p-value **faible** est le résultat **recherché** (elle confirme la structure spatiale), à l'inverse d'un test de normalité où une p-value faible signalerait un problème.

Les clusters locaux : LISA

Le Moran global dit qu'il y a de la structure, mais pas où. Le LISA (*Local Indicators of Spatial Association*) décompose l'indice global commune par commune et classe chacune selon son voisinage : *High-High* (commune dotée entourée de communes dotées, cœur d'un pôle), *Low-Low* (désert homogène), *High-Low* et *Low-High* (situations atypiques).

```
set.seed(42)
lisa <- localmoran(communes_l93$nb_formations, poids, zero.policy = TRUE)

# Récupération de la classification et du seuil de significativité
communes_l93$lisa_p <- lisa[, "Pr(z != E(Ii))"]

# Écart à la moyenne pour la commune et pour ses voisins
x         <- communes_l93$nb_formations
x_scale   <- as.numeric(scale(x))
lag       <- lag.listw(poids, x, zero.policy = TRUE)
lag_scale <- as.numeric(scale(lag))

communes_l93$cluster <- "Non significatif"
sig <- communes_l93$lisa_p < 0.05
communes_l93$cluster[sig & x_scale > 0 & lag_scale > 0] <- "High-High"
communes_l93$cluster[sig & x_scale < 0 & lag_scale < 0] <- "Low-Low"
communes_l93$cluster[sig & x_scale > 0 & lag_scale < 0] <- "High-Low"
communes_l93$cluster[sig & x_scale < 0 & lag_scale > 0] <- "Low-High"
```

```
table(communes_l93$cluster)
```

```
##
##        High-High         Low-High Non significatif
##               18               39             2594
```

L'analyse repère **18 communes en cluster High-High**, qui dessinent précisément les pôles universitaires : le cœur de l'agglomération caennaise (Caen, Hérouville-Saint-Clair, Ifs, Mondeville), l'agglomération rouennaise (Rouen, Mont-Saint-Aignan, Saint-Étienne-du-Rouvray, Sotteville-lès-Rouen), Le Havre, et un noyau ornais autour d'Alençon et Damigny. Elle identifie aussi **39 communes en Low-High**, des communes sans offre (ou presque) mais voisines des pôles. En revanche, aucune commune ne ressort en **Low-Low** ni en **High-Low** au seuil de 5 %, et cette absence s'explique : comme la valeur zéro est la norme régionale (97,1 % des communes), une commune vide entourée de communes vides ne s'écarte pas assez de la moyenne pour former un cluster significatif ; le « désert » est trop général pour être localement atypique.

Cartographier les regroupements

La carte des clusters LISA est l'aboutissement visuel de la démonstration. Les noms des communes clés (Caen, Hérouville, Rouen, etc.) sont désormais reportés sur la carte pour permettre une vérification visuelle immédiate.

```
couleurs <- c("High-High" = "#d7191c", "Low-Low" = "#2c7bb6",
              "High-Low" = "#fdae61", "Low-High" = "#abd9e9",
              "Non significatif" = "#f0f0f0")

tm_shape(communes_l93) +
  tm_polygons(fill = "cluster",
              fill.scale = tm_scale_categorical(values = couleurs),
              fill.legend = tm_legend(title = "Type de cluster"),
              col = "white", lwd = 0.1) +

  tm_shape(departements) +
  tm_borders(col = "black", lwd = 0.8) +
  tm_text(text = "NOM", size = 0.8, col = "black", fontface = "bold") +

  tm_shape(villes_a_labeliser) +
  tm_text(text = "NOM", size = 0.6, col = "black", fontface = "bold",
          shadow = TRUE, bg.color = "white", bg.alpha = 0.7) +

  tm_title("Regroupements spatiaux de l'offre de formation (LISA)") +
  tm_credits("Sources : Parcoursup 2024, IGN ; contiguïté Queen, p < 0,05")
```

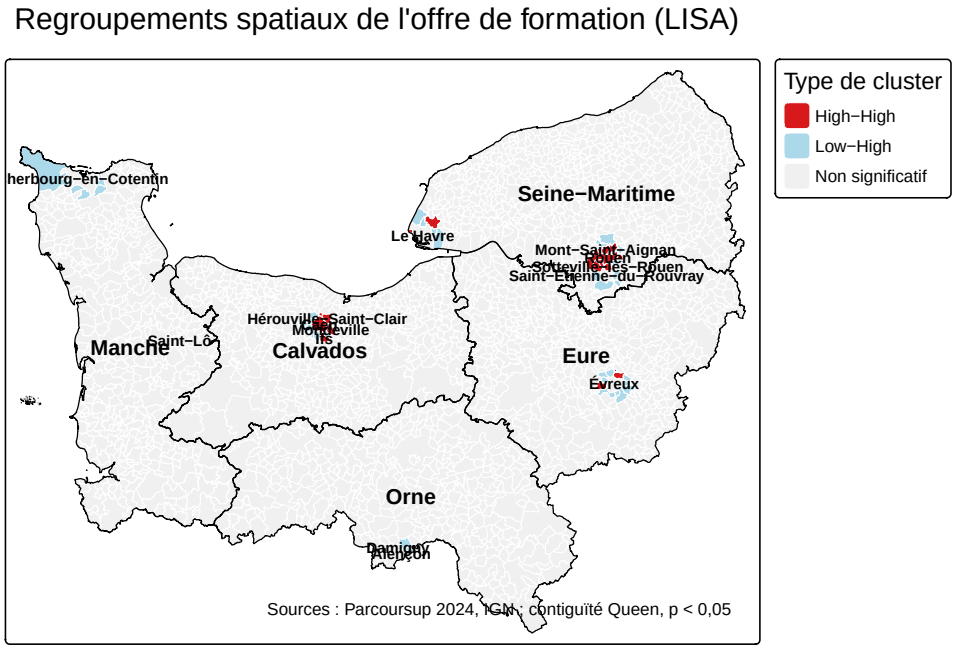


Figure 25: Clusters LISA de l'offre de formation, Normandie 2024

Les clusters rouges (High-High) confirment et localisent les pôles. La couronne bleu clair (Low-High) autour d'eux correspond aux communes périurbaines sans offre mais voisines des pôles : elles bénéficient potentiellement de la proximité, sans porter elles-mêmes de formation. On notera que les villes moyennes dotées mais isolées en milieu rural, comme Cherbourg, Saint-Lô ou Évreux, n'apparaissent pas en High-Low : leur voisinage uniformément vide ne suffit pas à rendre leur écart localement significatif au seuil de 5 %. Elles restent visibles sur les cartes de l'offre (figures 1 et 2), mais le LISA ne les distingue pas statistiquement.

Ce que révèle l'analyse

En reprenant la question posée en introduction, la réponse est claire : l'offre de formation supérieure en Normandie n'est pas équitablement répartie sur le territoire. Elle se polarise autour de trois métropoles, tandis que 97 % du territoire communal ne propose aucune formation locale.

Les cartes, le test de Moran et l'analyse LISA convergent vers ce même constat. L'offre se concentre sur Caen, Rouen et Le Havre, qui à elles seules captent près de la moitié des formations recensées — ce qui confirme l'hypothèse H1. Cette polarisation n'a rien d'aléatoire : l'indice de Moran (0,085 ; p = 0,005) est significatif, et les clusters High-High dessinent précisément le contour de ces trois pôles, validant ainsi l'hypothèse H2. La Normandie universitaire tient donc en quelques villes.

Une limite importante doit être posée. Ces données décrivent l'offre, c'est-à-dire les formations et leurs places. Elles ne décrivent pas la demande, c'est-à-dire les candidats et leur lieu de résidence. Une commune sans formation peut être proche d'un pôle. Ce n'est donc pas la présence locale qui compte le plus, mais la distance à parcourir. Pour mesurer les inégalités réelles, il faudrait croiser cette offre avec la localisation des candidats ou modéliser l'accessibilité par des zones tampon. Nous évoquerons ces pistes dans la conclusion générale.

Conclusion générale

La première partie de ce projet a présenté huit packages essentiels au traitement des données géospatiales. Dans l'application à la Normandie, nous avons sélectionné les trois qui répondent directement à notre question : sf pour préparer les données, tmap pour les cartographier et spdep pour tester leur structure spatiale. Cette chaîne de traitement est reproductible et peut être appliquée à d'autres régions ou à d'autres objets géographiques.

L'application à la Normandie avec les données Parcoursup 2024 donne des résultats clairs. L'offre est très concentrée. Les 615 formations recensées sont situées dans seulement 76 communes. Les trois agglomérations de Caen, Rouen et Le Havre concentrent 42,6 % des formations. 97 % des communes normandes n'en proposent aucune. Cette concentration n'est pas aléatoire. L'indice de Moran et les clusters LISA confirment une structure spatiale polarisée.

Notre travail porte sur l'offre, mais les inégalités d'accès dépendent aussi de la distance. Une commune sans formation n'est pas forcément isolée si elle est proche d'un pôle. À l'inverse, une commune dotée d'une seule formation n'offre pas forcément un large choix. Pour aller plus loin, il faudrait mesurer l'accessibilité réelle, par exemple avec des zones tampon ou des calculs de temps de trajet. Il faudrait aussi croiser ces données avec la localisation des candidats pour intégrer la dimension sociale.

Ce travail fournit un diagnostic utile pour les acteurs de l'orientation et de l'aménagement du territoire. Il montre que la géographie est un facteur clé de l'orientation post-bac. Comprendre où se trouvent les formations est un premier pas pour comprendre les contraintes qui pèsent sur certains parcours étudiants.

Bibliographie

Ouvrages et manuels de référence

Lovelace, R., Nowosad, J. et Muenchow, J. (2025). *Geocomputation with R* (2e édition). Londres : Chapman and Hall/CRC. Disponible en ligne : https://r.geocompx.org/

Pebesma, E. et Bivand, R. (2023). *Spatial Data Science: With applications in R*. Londres : Chapman and Hall/CRC. Disponible en ligne : https://r-spatial.org/book/

Apparicio, P. et Gelb, J. (2021). *Méthodes d'analyse spatiale*. Montréal : Les Presses de l'Université de Montréal. ISBN 978-2-7606-4291-6.

Accès territorial à l'enseignement supérieur

Thumerelle, P.-J. et Dumont, F. (1990). Géographie et inégalités dans l'accès à l'enseignement supérieur : le cas exemplaire de l'académie de Lille. *Espace, populations, sociétés*, 147–157. https://doi.org/10.3406/espos.1990.1424

Fabre, A. (2025). Inégalités territoriales d'accès à l'enseignement supérieur : quand la géographie façonne les parcours. *Focus CAE*, n°122, novembre 2025. Disponible sur : https://www.cae-eco.fr/

Couto, M.-P., Bugeja-Bloch, F. et Frouillou, L. (2021). Parcoursup : les prémices d'un accroissement de la stratification sociale et scolaire des formations du supérieur. *Agora débats/jeunesses*, 89, 23–38. https://doi.org/10.3917/agora.089.0023

Frouillou, L. (2025). Parcoursup et la reconfiguration des inégalités dans l'enseignement supérieur français. *Cairn*. https://www.cairn.info/

Sources de données

Institut Géographique National. (2024). *Admin Express — Produit contours*. Disponible sur : https://www.ign.fr/ Téléchargement direct : https://geoservices.ign.fr/adminexpress

Ministère de l'Enseignement supérieur et de la Recherche. (2024). *Données Parcoursup 2024*. Disponible sur : https://data.enseignementsup-recherche.gouv.fr/datasets/fr-esr-parcoursup-2024

Packages R (documentation officielle)

Pebesma, E. (2024). *sf: Simple Features for R*. https://r-spatial.github.io/sf/ CRAN : https://cran.r-project.org/package=sf

Wickham, H. et Chang, W. (2024). *ggplot2: Elegant Graphics for Data Analysis*. https://ggplot2.tidyverse.org/ CRAN : https://cran.r-project.org/package=ggplot2

Tennekes, M. (2024). *tmap: Thematic Maps* (v. 4.3+). https://r-tmap.github.io/tmap/ CRAN : https://cran.r-project.org/package=tmap

RStudio et Cheng, J. (2024). *leaflet: Create Interactive Web Maps with the JavaScript 'Leaflet' Library*. https://rstudio.github.io/leaflet/ CRAN : https://cran.r-project.org/package=leaflet

Hijmans, R. J. (2024). *terra: Spatial Data Analysis*. https://rspatial.org/terra/ CRAN : https://cran.r-project.org/package=terra

Pebesma, E. (2024). *stars: Spatiotemporal Arrays, Raster and Vector Data Cubes*. https://r-spatial.github.io/stars/ CRAN : https://cran.r-project.org/package=stars

Bivand, R. et Wong, D. W. S. (2024). *spdep: Spatial Dependence: Weighting Schemes, Statistics*. https://r-spatial.github.io/spdep/ CRAN : https://cran.r-project.org/package=spdep

Pebesma, E. et Graeler, B. (2024). *gstat: Spatial and Spatio-Temporal Geostatistical Modelling, Prediction and Simulation*. https://www.gstat.org/ CRAN : https://cran.r-project.org/package=gstat

Wickham, H., François, R., Henry, L. et Müller, K. (2024). *dplyr: A Grammar of Data Manipulation*. https://dplyr.tidyverse.org/ CRAN : https://cran.r-project.org/package=dplyr

Fondamentaux méthodologiques

Moran, P. A. P. (1948). The interpretation of statistical maps. *Journal of the Royal Statistical Society*, series B, 10, 243–251. https://doi.org/10.1111/j.2517-6161.1948.tb00012.x

Anselin, L. (1995). Local indicators of spatial association — LISA. *Geographical Analysis*, 27(2), 93–115. https://doi.org/10.1111/j.1538-4632.1995.tb00338.x

Tobler, W. R. (1970). A computer movie simulating urban growth in the Detroit region. *Economic Geography*, 46(Supplement), 234–240. https://doi.org/10.2307/143141

Openshaw, S. (1984). *The Modifiable Areal Unit Problem*. Norwich : Geo Books.

Annexe : Code des fonctions secondaires

Cette annexe regroupe le code complet des fonctions mentionnées dans le rapport. Tous les examples utilisent des jeux de données standard et reproductibles (nc du package sf, meuse du package sp) et sont conçus pour s'exécuter directement.

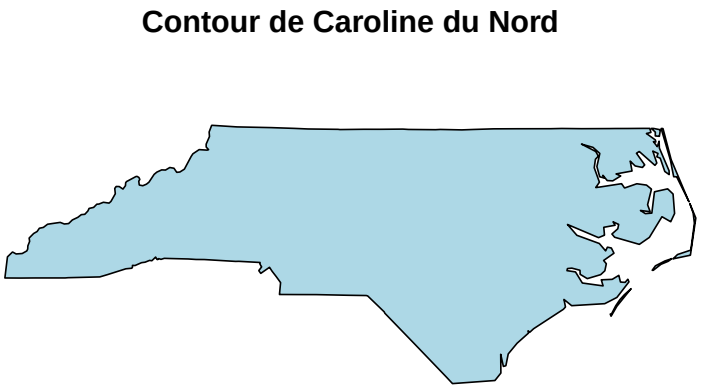
A.1 Fonctions sf secondaires

Fusion de polygones avec st_union()

```r
library(sf)
nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)

# Fusionner tous les comtés en une seule forme (enveloppe régionale)
contour <- st_union(nc)

# Afficher le résultat
plot(st_geometry(contour), main = "Contour de Caroline du Nord", col = "lightblue")
```



Buffer et autres opérations géométriques

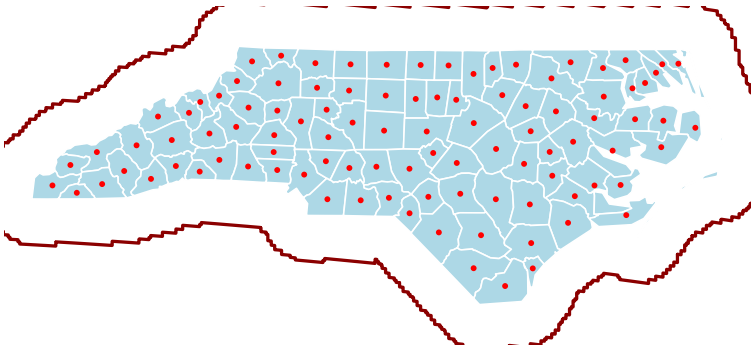
```r
library(sf)
nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)

# Centroïdes des comtés
centres <- st_centroid(nc)
contour <- st_union(nc)

# Zone tampon de 50 km autour de la région
tampon <- st_buffer(contour, dist = 50000)

# Visualiser
```

```r
plot(st_geometry(nc), col = "lightblue", border = "white")
plot(st_geometry(centres), add = TRUE, pch = 16, cex = 0.5, col = "red")
plot(st_geometry(tampon), add = TRUE, border = "darkred", lwd = 2)
```



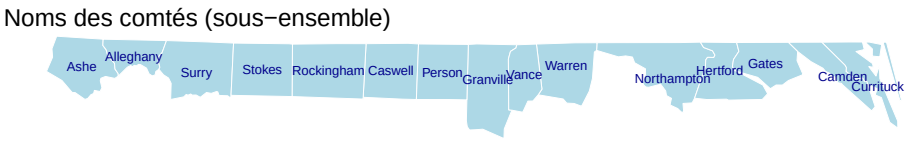
A.2 Fonctions ggplot2 secondaires pour la cartographie

Étiquetage avec geom_sf_text()

```r
library(ggplot2)
library(sf)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)

# Ajouter le nom des comtés (avec un sous-ensemble pour la lisibilité)
ggplot(nc[1:15,]) +
  geom_sf(fill = "lightblue", color = "white") +
  geom_sf_text(aes(label = NAME), size = 2.5, color = "darkblue") +
  labs(title = "Noms des comtés (sous-ensemble)") +
  theme_void()
```

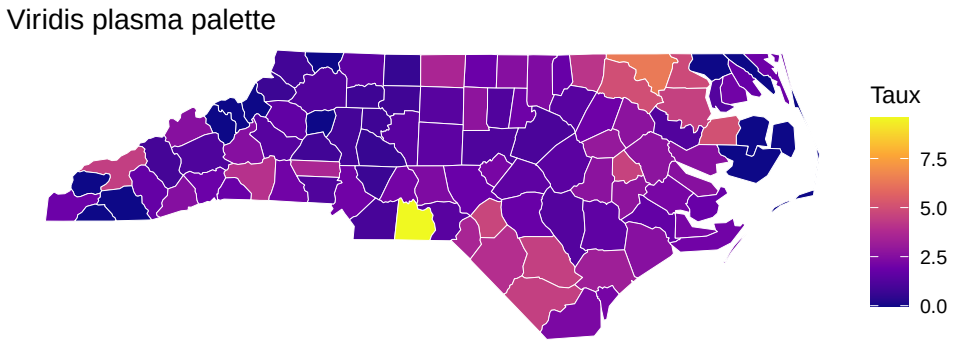


Palettes de couleurs avec viridis

```r
library(ggplot2)
library(sf)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
nc$taux <- 1000 * nc$SID74 / nc$BIR74

# Palette Viridis
ggplot(nc) +
  geom_sf(aes(fill = taux), color = "white", linewidth = 0.1) +
  scale_fill_viridis_c(option = "plasma", name = "Taux") +
  labs(title = "Viridis plasma palette") +
  theme_void()
```



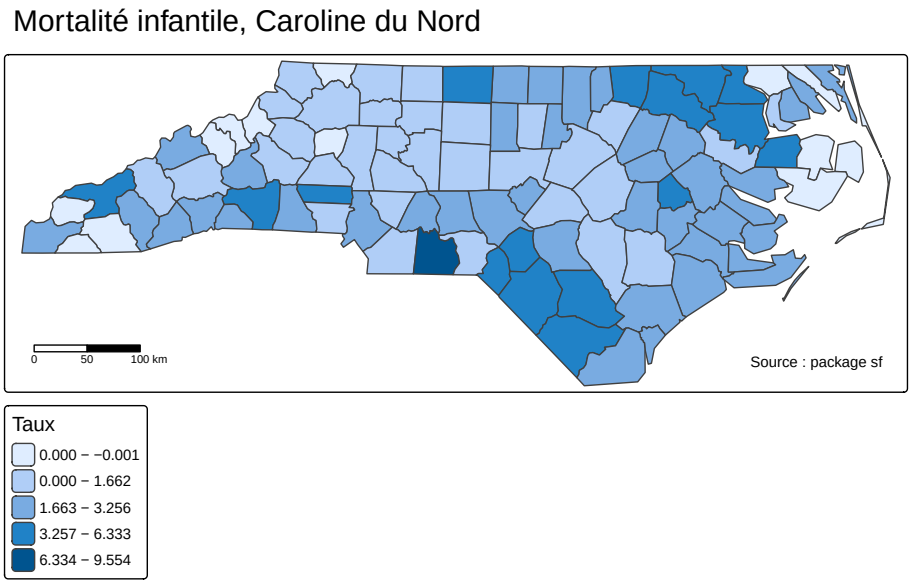
A.3 Fonctions tmap secondaires

Habillage complet avec tm_layout()

```
library(tmap)
library(sf)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
nc$taux <- 1000 * nc$SID74 / nc$BIR74

tm_shape(nc) +
  tm_polygons(fill = "taux",
              fill.scale = tm_scale_intervals(style = "jenks", n = 5),
              fill.legend = tm_legend(title = "Taux")) +
  tm_title("Mortalité infantile, Caroline du Nord") +
  tm_scalebar(position = c("left", "bottom")) +
  tm_credits("Source : package sf")
```

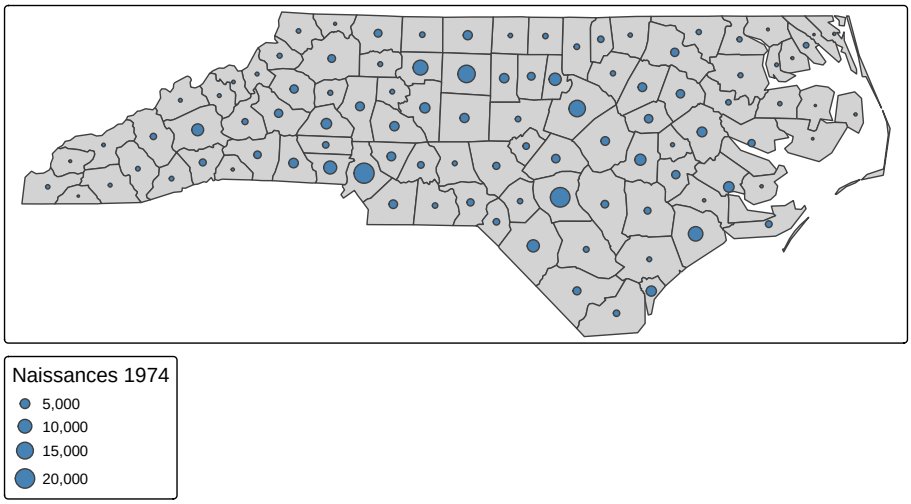


Cercles proportionnels

```
library(tmap)
library(sf)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)

tm_shape(nc) +
  tm_polygons(fill = "lightgrey", color = "white") +
  tm_shape(st_centroid(nc)) +
  tm_symbols(size = "BIR74", fill = "steelblue",
             size.legend = tm_legend(title = "Naissances 1974"))
```



A.4 Fonctions leaflet secondaires

Palette de couleurs et légende

```r
library(leaflet)
library(sf)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
nc$taux <- 1000 * nc$SID74 / nc$BIR74
nc_wgs <- st_transform(nc, crs = 4326)

# Créer une palette numérique
palette <- colorNumeric("RdYlGn", domain = nc_wgs$taux)

leaflet(nc_wgs) %>%
  addTiles() %>%
  addPolygons(fillColor = ~palette(taux),
              fillOpacity = 0.7,
              weight = 1,
              color = "white",
              popup = ~paste0(NAME, ": ", round(taux, 2))) %>%
  addLegend(position = "bottomright",
            pal = palette,
            values = ~taux,
            title = "Taux mortalité")
```

A.5 Fonctions terra

Statistiques de raster

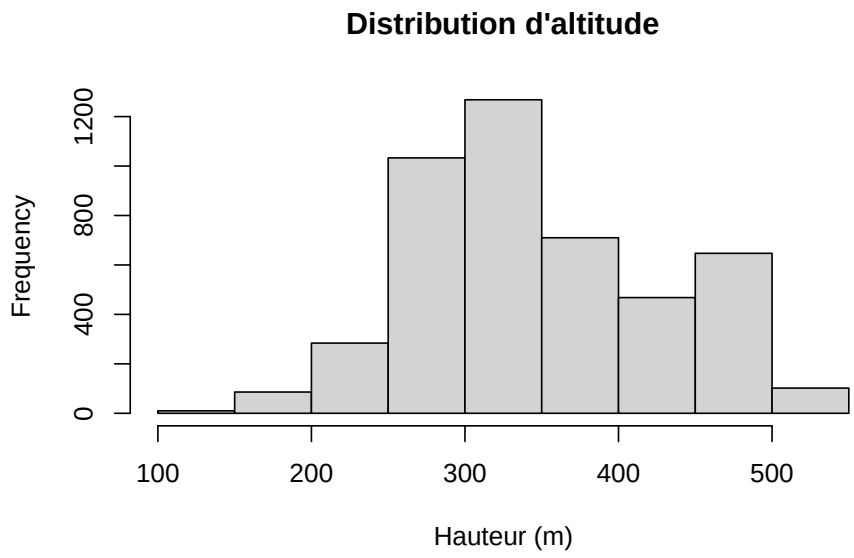
```r
library(terra)

# Raster d'altitude d'exemple (système de démonstration)
dem <- rast(system.file("ex/elev.tif", package = "terra"))

# Résumé statistique
summary(dem)
```

```
##    elevation
##  Min.   :141.0
##  1st Qu.:291.0
##  Median :333.0
##  Mean   :348.3
##  3rd Qu.:406.0
##  Max.   :547.0
##  NA's   :3942
```

```r
# Histogramme
hist(dem, main = "Distribution d'altitude", xlab = "Hauteur (m)")
```



A.6 Fonctions spdep

Voisinage Rook et k-plus proches voisins

```r
library(spdep)
library(sf)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
nc <- st_transform(nc, crs = 32617)

# Rook (4-voisinage)
```

```r
nb_rook <- poly2nb(nc, queen = FALSE)
summary(nb_rook)
```

```
## Neighbour list object:
## Number of regions: 100
## Number of nonzero links: 462
## Percentage nonzero weights: 4.62
## Average number of links: 4.62
## Link number distribution:
##
##  2  3  4  5  6  7  8  9
##  8 18 20 25 21  4  3  1
## 8 least connected regions:
## 4 21 45 56 77 80 90 99 with 2 links
## 1 most connected region:
## 39 with 9 links
```

Poids spatiaux alternatifs

```r
library(spdep)
library(sf)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
nc <- st_transform(nc, crs = 32617)

# Pondération standardisée (W)
nb_rook <- poly2nb(nc, queen = FALSE)
poids_w <- nb2listw(nb_rook, style = "W")

# Binary
poids_b <- nb2listw(nb_rook, style = "B")

# Afficher les résumés
summary(poids_w)
```

```
## Characteristics of weights list object:
## Neighbour list object:
## Number of regions: 100
## Number of nonzero links: 462
## Percentage nonzero weights: 4.62
## Average number of links: 4.62
## Link number distribution:
##
##  2  3  4  5  6  7  8  9
##  8 18 20 25 21  4  3  1
## 8 least connected regions:
## 4 21 45 56 77 80 90 99 with 2 links
## 1 most connected region:
## 39 with 9 links
##
## Weights style: W
## Weights constants summary:
##     n    nn  S0       S1       S2
## W 100 10000 100 46.85161 410.0147
```

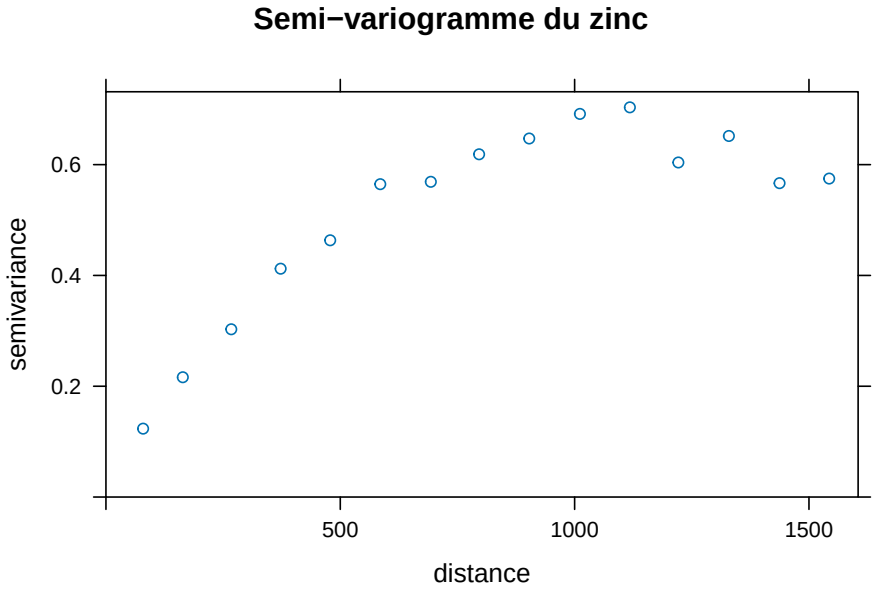
A.7 Gstat et géostatistique

Variogramme et diagnostic

```
library(gstat)
library(sf)

data(meuse, package = "sp")
meuse_sf <- st_as_sf(meuse, coords = c("x", "y"), crs = 28992)

# Variogramme omnidirectionnel
vario <- variogram(log(zinc) ~ 1, meuse_sf)
plot(vario, main = "Semi-variogramme du zinc")
```



Krigeage simple et universel

```
library(gstat)
library(sf)

data(meuse, package = "sp")
meuse_sf <- st_as_sf(meuse, coords = c("x", "y"), crs = 28992)

# Variogramme et modèle
vario <- variogram(log(zinc) ~ 1, meuse_sf)
modele <- fit.variogram(vario, model = vgm(1, "Sph", 900, 0.5))

# Créer une grille de prédiction
grille <- expand.grid(x = seq(178000, 182000, 1000),
                      y = seq(326000, 330000, 1000))
grille_sf <- st_as_sf(grille, coords = c("x", "y"), crs = 28992)

# Krigeage simple
krige_simple <- krige(log(zinc) ~ 1, meuse_sf, grille_sf, model = modele)
```

```
## [using ordinary kriging]
```

```
# Afficher les prédictions
plot(krige_simple["var1.pred"], main = "Zinc interpolé (log)",
     col = rev(heat.colors(100)))
```

